

**Efficient VLSI Design-Based Adaptive Pipelined
RISC-V Core for Low-Power Edge Devices**

LIEW MING HENG

UNIVERSITI SAINS MALAYSIA

2026

Efficient VLSI Design-Based Adaptive Pipelined RISC-V Core for Low-Power Edge Devices

by

LIEW MING HENG

**Thesis submitted in partial fulfilment of the requirements
for the degree of
Bachelor of Electronic Engineering with Honours**

July 2026

ACKNOWLEDGEMENT

First and foremost, I wish to express my sincere appreciation to my supervisor, Ts. Dr. Siti Fatimah Abd Rahman, for her invaluable advice and guidance throughout the development process. I would also like to thank my examiner, Mr. Zulfiqar Ali Bin Abdul Aziz for the constructive feedback that contributed significantly on improving the project's reliability. Furthermore, I extend my gratitude to the Final Year Project coordinators and the lecturers of the EEE449 course for organizing the seminars essential for the preparation of this thesis. Finally, I would like to show my deepest gratitude to all the lecturers at the School of Electrical and Electronic Engineering, Universiti Sains Malaysia, for providing the essential knowledge and technical skills throughout my Bachelor of Electronic Engineering studies. The expertise gained during this period was crucial to the successful design and implementation of the adaptive pipelined RISC-V processor.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	ii
TABLE OF CONTENTS.....	iii
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF SYMBOLS	x
LIST OF ABBREVIATIONS	xi
LIST OF APPENDICES	xiii
ABSTRAK	xiv
ABSTRACT.....	xvi
CHAPTER 1 INTRODUCTION.....	1
1.1 Research Background.....	1
1.2 Problem Statement	3
1.3 Research Objectives	4
1.4 Research Scope	5
1.5 Research Justification.....	6
1.6 Thesis Organization.....	7
CHAPTER 2 LITERATURE REVIEW.....	10
2.1 Introduction	10
2.2 The RISC-V Microarchitectural Paradigm.....	11
2.2.1 The RV32I Base Integer Instruction Set	11
2.3 Instruction Pipelining and Hazard Management.....	13
2.3.1 Frequency vs. Overhead Trade-Off.....	14
2.3.2 Data Dependencies and the Hardware Cost of Mitigation.....	14
2.3.3 Control Hazards and Flush Penalties	17
2.4 Low-Power VLSI Design and Adaptive Architectures.....	19

2.4.1	Dynamic Frequency and Voltage Scaling (DVFS)	19
2.4.2	RTL Level Optimization	20
2.4.2(a)	Instruction-Aware Clock Gating.....	20
2.4.2(b)	Transparent Register Modes and Pipeline Stage Unification (PSU)	21
2.5	Review of Related RISC-V Implementations	21
2.5.1	Thematic Comparison of Core Architectures.....	21
2.5.2	Verification Methodologies: Testbenches vs. Compliance Frameworks.....	22
2.6	Summary and Identification of Research Gaps.....	24
2.6.1	Summary of Findings	24
2.6.2	Strengths and Limitations of Existing Research	25
2.6.3	Identification Research Gaps	25
2.6.3(a)	Rigidity in Pipeline Depth	26
2.6.3(b)	Penalty of Static Hazard Mitigation	26
2.6.3(c)	Lack of Industrial-Grade Verification for Adaptive Cores	26
2.6.4	Project Justification	27
CHAPTER 3 METHODOLOGY		28
3.1	Introduction and Project Workflow.....	28
3.2	Development and Hardware Specification.....	30
3.2.1	Hardware Prototyping Platform	30
3.2.2	Hybrid Development Environment (WSL2)	32
3.2.3	Synthesis and Simulation Toolchain	32
3.3	Phase 1: Baseline Single-Cycle Architecture	33
3.3.1	Control Logic Domain	35
3.3.1(a)	Main Control Unit.....	35
3.3.1(b)	ALU Control Logic.....	36

3.3.2	Datapath Logic Domain	36
3.3.2(a)	Arithmetic Logic Unit (ALU).....	37
3.3.2(b)	Immediate Generator	38
3.3.2(c)	Program Counter (PC)	38
3.3.3	Storage and Memory Logic Domain.....	39
3.3.3(a)	Register File.....	39
3.3.3(b)	Control and Status Register (CSR) Unit.....	41
3.3.3(c)	Instruction and Data Memory Interface.....	41
3.4	Phase 2: Transition to the Fixed 5-Stage Pipeline.....	43
3.4.1	Pipelining the Datapath (Structural Segmentation).....	43
3.4.2	Hazard Mitigation Strategies.....	44
3.5	Phase 3: The Adaptive Architecture.....	47
3.5.1	Pipeline Stage Unification (PSU).....	48
3.5.2	Glitch-Free Dual-Clock Domain Architecture.....	50
3.5.3	Combinational Power Gating (Operand Isolation).....	52
3.6	System Verification and Implementation Strategy	54
3.6.1	Verification Methodology: Modular to System-Level.....	54
3.6.2	FPGA Synthesis and Timing Closure	57
3.6.3	Empirical Power Estimation Methodology	58
3.7	Chapter Summary.....	59
CHAPTER 4 RESULTS AND DISCUSSION.....		61
4.1	Introduction	61
4.2	Baseline Performance of Single-Cycle RV32I Processor	61
4.2.1	Synthesis and Hardware Resource Utilization Profile	62
4.2.2	Timing and Propagation Path Analysis.....	64
4.2.3	Power Characterization and Dissipation Analysis	65
4.3	Baseline Performance of 5-Stage Pipeline RV32I Processor (Phase 2).....	68

4.3.1	Hardware Resource Utilization and Fabric Overhead.....	68
4.3.2	Static Timing Analysis and Frequency Closure.....	69
4.3.3	Power Characterization and Dissipation Profile	70
4.3.4	Architectural Compliance and Pipeline Hazard Penalties.....	71
4.4	Performance Evaluation of Software-Reconfigurable 3/5-Stage Adaptive Processor (Phase 3)	75
4.4.1	Control Logic Footprint and Structural Area Overhead.....	76
4.4.2	Workload Execution Throughput Across Different Modes	77
4.4.3	Workload-Aware Dynamic Power Reduction.....	79
4.5	Comprehensive Architectural Comparison and Efficiency Metrics.....	80
4.5.1	Cross-Architectural Efficiency Evaluation	80
4.5.2	Workload Suitability and Scenario Analysis	83
4.6	Hardware Implementation and System-on-Chip (SoC) Integration.....	86
4.6.1	SoC Architecture and Peripheral Bus Routing.....	86
4.6.2	Physical Board Demonstration.....	88
4.7	Chapter Summary.....	90
CHAPTER 5 CONCLUSION AND FUTURE WORK		91
5.1	Summary of Achieved Research Objectives.....	91
5.2	Research Contributions and Key Findings.....	92
5.3	Future Work	94
REFERENCES.....		96
APPENDICES		

LIST OF TABLES

	Page
Table 2.1: Meaning and Purpose of R, I, S, B, U, J Types in RISC-V [5].	12
Table 2.2: Thematic Comparison of Prominent Cores [8] [12]	22
Table 2.3: Strengths and Limitations of Existing Research.....	25
Table 3.1: Summary of Project Development and Verification Tools.....	33
Table 4.1: Single-Cycle Core Hardware Resource Utilization	62
Table 4.2: Single-Cycle Core Timing and Performance Profile	64
Table 4.3: Baseline Single-Cycle Core Power Dissipation Parameters	66
Table 4.4: Static 5-Stage Pipeline Resource Utilization	68
Table 4.5: Static 5-Stage Pipeline Timing and Performance Profile	69
Table 4.6: Static 5-Stage Pipeline Power Dissipation Summary	71
Table 4.7: 5-Stage Pipeline Hazard Penalty and Throughput Breakdown.....	72
Table 4.8: Adaptive Multi-Stage Core Hardware Resource Utilization	76
Table 4.9: Adaptive Core Throughput Profiling Across Reconfiguration Modes	77
Table 4.10: Adaptive Core Timing and Performance Profile	78
Table 4.11: Dynamic Power Dissipation Comparison Across Processing Modes.....	79
Table 4.12: Cross-Architectural Figure of Merit Comparison.....	81

LIST OF FIGURES

	Page
Figure 1.1: Standard 5-stage Pipelining Datapath.....	1
Figure 2.1.: 32-bit RISC-V Instruction Formats (R, I, S, B, U, J format diagram) [5].	13
Figure 2.2: Block Diagram of a 5-Stage Pipelined RISC-V Core featuring explicit data forwarding paths and a Hazard Unit. Adapted from [9].....	16
Figure 2.3: Block Diagram of an optimized branch history predictor for RISC-V processors. Adapted from Jin et al. [7].	18
Figure 3.1: Three-Phase Iterative Design Flow	29
Figure 3.2: Top View of CertusPro-NX Versa Evaluation Board	31
Figure 3.3: Microarchitecture of the Baseline Single-Cycle RV32I SoC.....	34
Figure 3.4: I/O Ports for Control Unit Module	35
Figure 3.5: I/O Ports for ALU Control Module	36
Figure 3.6: I/O Ports for ALU Module	37
Figure 3.7: I/O Ports for Immediate Generator Module	38
Figure 3.8: I/O Ports for PC Module.....	38
Figure 3.9: I/O Ports for Register File Module	39
Figure 3.10: General Purpose Registers in RISC-V Specifications	40
Figure 3.11: I/O Ports for CSR Unit Module.....	41
Figure 3.12: I/O Ports for Instruction ROM Module	42
Figure 3.13: I/O Ports for Data RAM Module.....	42
Figure 3.14: Structural Segmentation of the 5-Stage RV32I Pipelined Datapath.....	44
Figure 3.15: Read-After-Write (RAW) Data Hazard.....	45
Figure 3.16: Load-Use Stall Hazard	46
Figure 3.17: Control Hazard with Pipeline Flushing	47

Figure 3.18: Microarchitecture of the Adaptive Core.....	49
Figure 3.19: Adaptive Core operating in 3-Stage Mode.....	49
Figure 3.20: Finite State Machine (FSM) for the Glitch-Free Clock Multiplexing...	51
Figure 3.21: Timing Diagram of the Glitch-Free Clock Transition.....	52
Figure 3.22: Operand Isolation Architecture	53
Figure 3.23: Verification Hierarchy and Implementation Flow.....	55
Figure 3.24: Lattice Design Constraint (.ldc) File Configuration.....	57
Figure 3.25: Lattice Power Calculator Environment.	59
Figure 4.1: Proportional Power Dissipation Components of the Baseline Single- Cycle Core.....	67
Figure 4.2: 100% Compliance of the 5-Stage Pipeline Architecture with the RISCOF RV32I Test Suite against the Spike Reference Model.....	72
Figure 4.3: Realized MIPS Throughput Comparison across Different Instruction Test Workloads.	75
Figure 4.4: Architectural Efficiency Comparison across four hardware configurations.....	83
Figure 4.5: Power Consumption of both 5-Stage and Adaptive cores across variable workload.....	84
Figure 4.6: Relational Average Power Dissipation Models across Variable Active Duty Cycle	85
Figure 4.7: System-on-Chip (SoC) Physical Memory Map.....	87
Figure 4.8: Physical Hardware Demonstration of the Lattice CertusPro-NX FPGA board.....	89

LIST OF SYMBOLS

T_{exec}	Total execution time (seconds)
IC	Instruction count (total number of instructions executed)
N	Total number of instructions
CPI	Average cycles per instruction
f	Operating clock frequency (Hz)
f_{max}	Maximum operating frequency (Hz)
α	Switching activity factor (dimensionless)
C	Effective load capacitance (Farads)
C_l	Load capacitance (Farads)
C_L	Total capacitive load (Farads)
$C_{latches}$	Capacitance contribution of pipeline latch registers (Farads)
C_{logic}	Capacitance contribution of combinational logic units (Farads)
V	Supply voltage (Volts)
V_{dd}	Power supply voltage (Volts)
P_{dyn}	Dynamic power consumption (Watts)
$P_{dynamic}$	Dynamic power consumption (Watts) - alternate notation
P_{total}	Total power consumption including latches and logic units (Watts)
E_{dyn}	Dynamic energy per instruction (nJ/instruction)
D	Active duty cycle ratio (dimensionless; $0 \leq D \leq 1$)
$P_{avg(STATIC)}$	Average dynamic power dissipation of the static 5-stage pipeline (Watts)
$P_{avg(ADAPTIVE)}$	Average dynamic power dissipation of the adaptive reconfigurable core (Watts)

LIST OF ABBREVIATIONS

AIoT	Artificial Intelligence of Things
ALU	Arithmetic Logic Unit
APB	Advanced Peripheral Bus
ASIC	Application-Specific Integrated Circuit
BSS	Block Started by Symbol (program data segment)
BTB	Branch Target Buffer
CISC	Complex Instruction Set Architecture
CMOS	Complementary Metal-Oxide-Semiconductor
CPI	Cycles Per Instruction
CPU	Central Processing Unit
CSR	Control and Status Register
DFF	D Flip-Flop
DSP	Digital Signal Processing / Digital Signal Processor
DUT	Design Under Test
DV	Design Verification
DVFS	Dynamic Voltage and Frequency Scaling
EBR	Embedded Block RAM
EDP	Energy-Delay Product
EX	Execution Stage
FD-SOI	Fully Depleted Silicon-On-Insulator
FPGA	Field Programmable Gate Array
FSM	Finite State Machine
GPIO	General Purpose Input/Output
GNU	GNU's Not Unix (open-source toolchain)
GUI	Graphical User Interface
HDL	Hardware Description Language
IC	Integrated Circuit
ID	Instruction Decode Stage
IF	Instruction Fetch Stage
IPC	Instructions Per Clock
ISA	Instruction Set Architecture
LSE	Lattice Synthesis Engine

LUT	Look-Up Table
LUT-RAM	Look-Up Table used as Distributed RAM
MEM	Memory Access Stage
MIPS	Millions of Instructions Per Second
MMIO	Memory-Mapped Input/Output
NOP	No-Operation (instruction)
PC	Program Counter
PDP	Power-Delay Product
PHT	Pattern History Table
PLL	Phase-Locked Loop
PSU	Pipeline Stage Unification
RAM	Random Access Memory
RISC	Reduced Instruction Set Architecture
RISCOF	RISC-V Compliance Framework
ROM	Read-Only Memory
RTL	Register Transfer Level
RV32I	RISC-V 32-bit Base Integer Instruction Set
SLA	Arithmetic Left Shift
SLBHP	Simplified Local Branch History Predictor
SLL	Logical Left Shift
SoC	System-on-Chip
SRA	Arithmetic Right Shift
SRL	Logical Right Shift
SSR	Stall Scheme Reducing
STA	Static Timing Analysis
UART	Universal Asynchronous Receiver-Transmitter
VCD	Value Change Dump
VLSI	Very Large-Scale Integration
WB	Write-Back Stage
WSL	Windows Subsystem for Linux

LIST OF APPENDICES

Appendix A WSL2 Environment Setup and RISCOF Toolchain Installation

Appendix B Open-Source Repository

TERAS RISC-V TALIAN PAIP PENYESUAIAN BERASASKAN REKA BENTUK VLSI YANG CEKAP UNTUK PERANTI PINGGIR BERKUASA RENDAH

ABSTRAK

Projek ini memfokuskan kepada pembangunan teras pemproses RISC-V RV32I berkuasa rendah dan adaptif yang disasarkan untuk aplikasi pengkomputeran pinggir yang mempunyai kekangan kuasa. Seni bina saluran paip tetap konvensional sering mengalami aktiviti pensuisan yang lewah dan hasil darab tenaga-lengah yang tidak cekap apabila memproses beban kerja yang pelbagai. Bagi menangani batasan ini, projek ini membentangkan reka bentuk seni bina berevolusi, yang berkembang daripada teras kitaran tunggal asas kepada saluran paip 5-tahap tetap, dan akhirnya memperkenalkan mekanisme saluran paip adaptif yang boleh dikonfigurasi semula melalui perisian. Mekanisme ini mengkonfigurasi semula kedalaman saluran paip secara dinamik antara mod kecekapan 3-tahap dan mod prestasi 5-tahap melalui penyatuan tahap saluran paip pada peringkat perkakasan, penghalaan laluan data adaptif, dan get kuasa daftar. Reka bentuk ini dibangunkan melalui pemodelan Peringkat Pemindahan Daftar (RTL) dalam Verilog, simulasi kitaran tepat, dan pelaksanaan fizikal pada FPGA Lattice CertusPro-NX. Pengesahan fungsian telah dilaksanakan dengan teliti menggunakan rangka kerja ujian seni bina RISCOF, memastikan 100% pematuhan Seni Bina Set Arahan (ISA) terhadap model rujukan Spike merentas semua set ujian standard dalam spesifikasi RV32I dan Zicsr. Prestasi seni bina mikro dan sampul kuasa dinilai menggunakan set ujian penanda aras mikro *C bare-metal* 4-fasa tersuai, yang memetakan profil operasi yang berbeza: intensif-ALU, intensif-storan, intensif bahaya muat-guna, dan pepohon pelaksanaan kawalan berat. Dengan melaksanakan litar pemultipleks jam dwi-domain tanpa gangguan, Sistem-atas-Cip (SoC) adaptif tersebut mencapai penutupan pemasaan yang stabil

pada frekuensi operasi 50 MHz dalam mod 5-tahap dan 20 MHz dalam mod 3-tahap. Melalui anotasi surihan Lompakan Perubahan Nilai (VCD) dan pengesahan fizikal, hasil eksperimen menunjukkan pengurangan purata penggunaan kuasa dinamik sebanyak 39.51% apabila beralih kepada mod kecekapan 3-tahap semasa kitaran tugas IoT tipikal. Penemuan ini berjaya merapatkan jurang antara penskalaan saluran paip teori dan realiti fizikal, mengesahkan penyesuaian seni bina mikro pada masa jalanan sebagai strategi pengoptimuman yang sangat praktikal dan cekap kuasa untuk sistem terbenam generasi akan datang.

EFFICIENT VLSI DESIGN-BASED ADAPTIVE PIPELINED RISC-V CORE FOR LOW-POWER EDGE DEVICES

ABSTRACT

This project focuses on the development of an adaptive, low-power RV32I RISC-V processor core targeted for power-constrained edge computing applications. Conventional fixed-pipeline architectures often incur redundant switching activity and inefficient energy-delay products when processing variable workloads. To address these limitations, this project presents an evolutionary architectural design, progressing from a baseline single-cycle core to a fixed 5-stage pipeline, and ultimately introducing a software-reconfigurable adaptive pipelining mechanism. This mechanism dynamically reconfigures the effective pipeline depth between a 3-stage efficiency mode and a 5-stage performance mode through hardware-level pipeline stage unification, adaptive datapath routing, and register power-gating. The design progressed through Register Transfer Level (RTL) modeling in Verilog, cycle-accurate simulation, and physical deployment on a Lattice CertusPro-NX FPGA. Functional verification was rigorously executed using the RISCOF architectural test framework, ensuring 100% Instruction Set Architecture (ISA) compliance against the Spike reference model across all standard test suites within the RV32I and Zicsr specifications. Microarchitectural performance and power envelopes were assessed using a custom 4-phase bare-metal C micro-benchmark suite, which mapped distinct operational profiles: ALU-intensive, store-intensive, load-use hazard-intensive, and control-heavy execution trees. By implementing a glitch-free dual-domain clock multiplexer circuit, the adaptive System-on-Chip (SoC) achieved stable timing closure at operational frequencies of 50 MHz in the 5-stage mode and 20 MHz in the 3-stage

mode. Through Value Change Dump (VCD) trace annotation and physical validation, experimental results demonstrate a 39.51% reduction in average dynamic power consumption when transitioning to the 3-stage efficiency mode during typical IoT duty cycles. These findings successfully bridge the gap between theoretical pipeline scaling and physical reality, validating runtime microarchitectural adaptation as a highly feasible, power-efficient optimization strategy for next-generation embedded systems.

CHAPTER 1

INTRODUCTION

1.1 Research Background

The expansion of the Artificial Intelligence of Things (AIoT) and edge computing has placed stringent energy constraints on modern silicon design. Historically, microprocessor development focused on raw computational throughput, driving the shift from Complex Instruction Set Computer (CISC) architectures to the streamlined Reduced Instruction Set Computer (RISC) philosophy [1]. By utilizing fixed-length instructions, RISC architecture reduced hardware complexity and enabled efficient deep instruction pipelining. As shown in Figure 1.1, a standard 5-stage pipeline allows for instruction overlap, targeting an ideal Cycles Per Instruction (CPI) of 1, which has established RISC as the preferred architecture for embedded systems.

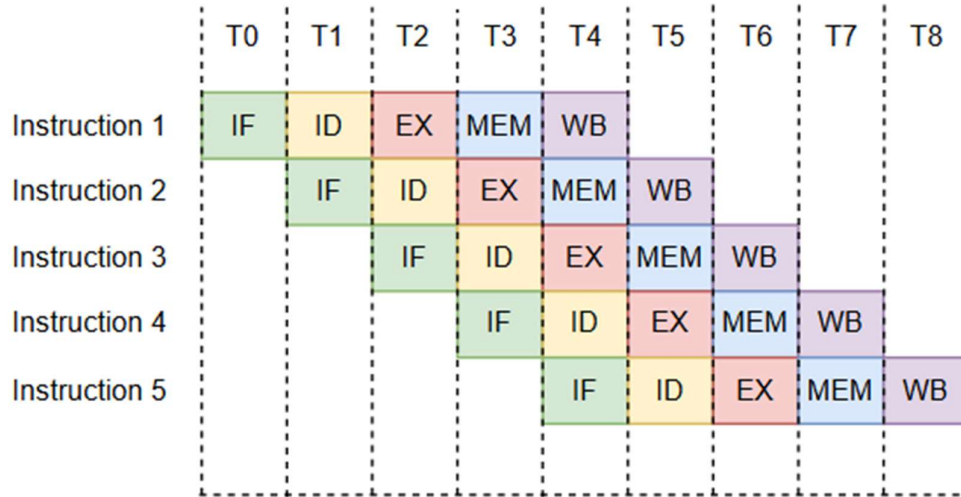


Figure 1.1: Standard 5-stage Pipelining Datapath

However, deep sub-micron design optimization is limited by the trade-off between computational throughput and power efficiency. The efficiency of a processor is governed by the classic CPU performance equation:

$$T_{exec} = IC \times CPI \times \frac{1}{f} \quad (1.1)$$

where T_{exec} represents execution time, IC is the instruction count, CPI is the average cycles per instruction, and f is the operating clock frequency. While architects traditionally maximize frequency and pipeline depth to reduce execution time, the dynamic power consumption (P_{dyn}) of CMOS-based logic is simultaneously governed by the relation:

$$P_{dyn} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.2)$$

Where α represents the switching activity factor, C is the effective load capacitance, and V is the supply voltage. In conventional microprocessor architectures, these variables remain rigidly fixed. Even during periods of idle execution, pipeline stalls, or low-priority tasks, the entire datapath infrastructure continues to oscillate at the operational frequency. This results in significant power dissipation through unnecessary toggling with massive combinational logic blocks. Consequently, there is a critical need to transition from static design methodologies to adaptive architectures that are capable of dynamically modulating both frequency and switching activity based on real-time workload demand.

Despite these RISC's technical advantages, the proprietary nature of traditional ISAs has often restricted deep microarchitectural customization [2]. In this context, RISC-V Instruction Set Architecture (ISA) which was developed by the Computer Science Department at the University of California, Berkeley in 2010 represents a transformative shift [2]. As an open-source ISA, its modular structure which is centered on the base integer set (RV32I) allows designers to tailor hardware specifically for targeted applications. This flexibility is critical for Artificial Intelligence of Things (AIoT) and edge computing, where devices must operate under stringent power, area, and thermal limitations.

1.2 Problem Statement

As workloads on Artificial Intelligence of Things (AIoT) and edge devices grow in complexity, the design of efficient RISC-V processors faces a critical bottleneck in balancing high instruction throughput with strict energy constraints. Instruction pipelining is the standard architectural technique used to improve performance by overlapping execution stages. However, the selection of fixed pipeline depth presents a rigid microarchitectural trade-off. While deeper pipelines enable higher clock frequencies by shortening the critical path, they significantly inflate the total capacitive load due to the increased number of intermediate pipeline registers [21]. Furthermore, deeper pipelines inherently exacerbate hardware conflicts, specifically Read-After-Write (RAW) data dependencies and control hazards.

To resolve these hazards, current research relies heavily on static microarchitectural additions such as sophisticated branch predictors and extensive data forwarding networks. While these mechanisms minimize pipeline stalls, they introduce severe power and area penalties. For instance, advanced bypass networks like the Stall Scheme Reducing (SSR) architecture successfully eliminate load-use stalls but actively increases dynamic power consumption by 8.4% and core area by 6.4% [8]. Similarly, complex branch predictors inflate logic utilization, adding constant static power overheads regardless of the workload [7]. In fixed deep pipelines, the hardware remains fully active even during simple arithmetic phases that do not require deep segmentation. This leads to redundant clock switching and costly branch misprediction flushes which waste the dynamic energy previously expended to fetch and decode discarded instructions.

Existing fixed-pipeline RISC-V architectures lack the elasticity to adapt to varying workload behaviors in real-time. A static 5-stage core provides peak

performance but consumes unnecessary energy for simple tasks, whereas a rigid 3-stage design limits maximum instruction throughput for complex computational loads. Theoretical solutions, such as Pipeline Stage Unification (PSU), suggest that dynamically scaling pipeline depth can drastically reduce power through bypassing redundant registers [12]. However, practical implementations of adaptive 3-to-5 stage scaling in open-source RISC-V cores remain largely unexplored.

Crucially, most academic proposals for pipelined processors rely on simple and directed HDL testbenches rather than using RISCOF [3]. This leaves them vulnerable to silent data corruption, particularly during pipeline transition states. There is also a distinct lack of physical, hardware-validated power profiling on Field Programmable Gate Arrays (FPGAs) to prove this theoretical energy reduction. Consequently, there is critical need for a fully verified adaptive pipelining mechanism that can dynamically scale its effective pipeline depth to eliminates redundant switching activity, achieves stable timing closure and optimizes the Power-Delay Product (PDP) for resource-constrained edge devices.

1.3 Research Objectives

The objectives of this work are:

1. To design a runtime-reconfigurable 3/5-stage RV32I processor with a minimum 50% reduction in dynamic power consumption during 3-stage efficiency mode relative to a fixed 5-stage baseline.
2. To verify the functional correctness of both pipeline configurations through cycle-accurate simulation by achieving 100% ISA compliance across all 39 RISCOF architectural tests against the Spike reference model.

3. To implement the System-on-Chip (SoC) on a Lattice CertusPro-NX FPGA, achieving stable timing closures of 50 MHz (5-stage) and 20 MHz (3-stage) while demonstrating a 40% average dynamic energy reduction under typical IoT duty cycles.

1.4 Research Scope

This research focuses on the microarchitectural design, functional verification, and FPGA-based evaluation of an adaptive RV32I processor core. Regarding the Instruction Set Architecture (ISA), the processor implements the unprivileged RV32I Base Integer Instruction Set. To maintain a lightweight architectural footprint optimized for configuration scaling, standard extensions such as hardware multiplication and division ('M'), atomic instruction ('A'), and compressed instructions ('C') are excluded. However, the design includes a specialized base-metal subset of the 'Zicsr' extension to implement basic execution counters and the configuration register. Full machine-mode privileged trap handling, exception processing, and vectored interrupt routines fall outside the scope of this work.

In terms of microarchitectural design, the dynamic reconfiguration capabilities are strictly bound to runtime shifting between a 3-stage efficiency mode and a 5-stage performance mode. This structural alteration is software-controlled via CSR register writes that utilize an internal clock-switching finite state machine (FSM) alongside dynamic routing logic. Advanced architectural features such as out-of-order execution, superscalar execution pipelines, and hardware branch prediction tables are explicitly excluded from this study.

The verification framework is structured to ensure functional correctness through architectural simulation compliance testing. This is executed using the RISC-

V Compatibility Framework (RISCOF) evaluated against the Spike ISA reference model. To guarantee that the processor meets the core RV32I requirements, the design must successfully pass all 39 standard architectural test suites from the official compliance set within a Windows Subsystem for Linux (WSL2) environment. Formal verification methods and hardware-level security audits are not included in this scope.

Finally, regarding the hardware platform, design tools, and performance metrics, all prototyping, placement, routing, and static timing closure analyses are performed for the Lattice CertusPro-NX FPGA board using the Lattice Radiant software suite. Functional and cycle-accurate simulations are executed using QuestaSim, which is directly linked within the Lattice Radiant environment. Power evaluation is specifically confined to post-route gate-level software estimation. This is achieved by extracting switching activity metrics from cycle-accurate simulations via Value Change Dump (VCD) trace files and importing them into the Lattice Power Calculator tool. Consequently, direct physical power measurement falls outside the scope of this research.

1.5 Research Justification

The justification for this research is driven by the critical demand for energy-delay optimization in battery-operated edge computing and artificial intelligence of things (AIoT) systems. Conventional fixed-pipeline architectures present an inflexible compromise between peak throughput and static power overhead. A fixed 5-stage pipeline incurs high dynamic power consumption due to continuous clock distribution and flip-flop toggling across dense pipeline registers, even when executing simple instruction streams or enduring stall conditions. Conversely, a shallow or single-cycle architecture eliminates register stage overhead but introduces severe combinational

path delays, depressing the maximum operational frequency (F_{max}). By implementing a software-programmable adaptive microarchitecture capable of runtime configuration between a unified 3-stage mode and a decoupled 5-stage mode, this research demonstrates a direct method for dynamic workload-to-hardware matching. When executing control-heavy instructions, the core utilizes its 5-stage configuration at 50 MHz to process data with high throughput. During low-compute intervals, the system drops into 3-stage mode at 20 MHz via a custom glitch-free clock multiplexer, bypassing intermediate register barriers while lowering the active switching activity factor (α). This approach minimizes the energy-per-instruction footprint, directly extending the operational lifespan of power-constrained edge nodes without sacrificing the performance capabilities.

From an industrial perspective, this research aligns directly with the strategic shift within the Malaysia semiconductor ecosystem from semiconductor packaging and testing toward high-value front-end Integrated Circuit (IC) design. This project provides a practical case study in executing industry-standard digital design methodologies, specifically focusing on RTL modelling, static timing closure analysis within the Lattice Radiant suite, and cycle-accurate co-simulation in QuestaSim. Furthermore, by utilizing the formal RISCOF framework for architectural compliance verification against a golden reference model, this research ensuring 100% specification correctness making the verification strategies developed in this work highly relevant to current industrial digital design practices.

1.6 Thesis Organization

The thesis is structured into five comprehensive chapters to provide a logical flow from theoretical motivation to physical implementation and evaluation.

Chapter 1 presents the introduction of the project, establishing the research framework by detailing the background of the RISC-V architecture. It defines the power-performance trade-offs and structural bottlenecks associated with fixed-depth processing pipelines, while delineating the specific research objectives, architectural scope, and technical justifications for developing the software -reconfigurable adaptive core.

In Chapter 2, a critical literature review of existing microprocessor architecture is presented. This review explores the evolution of instruction pipelining, common hazard mitigation techniques, and state-of-the-art research regarding open-source RISC-V cores.

In Chapter 3, the design methodology and technical implementations of the System-on-Chip (SoC) are explained through a three-phase iterative engineering flow. This progression includes the initial development of a functional single-cycle baseline core, its structural segmentation into a fixed 5-stage pipeline with hazard-mitigation logic, and its final transformation into a runtime-configurable 3/5-stage adaptive core. The chapter explicitly documents the design of custom datapath multiplexers, write-buffer barrier registers, and a glitch-free dual-domain clock multiplexer, alongside the verification methodology utilizing QuestaSim and the RISCOF framework within a WSL2 environment.

In Chapter 4, the evaluation results and discussions are presented. This chapter documents the physical synthesis, placement, and routing of each microarchitectural phase mapped onto the 28nm FD-SOI Lattice CertusPro-NX FPGA fabric using Lattice Radiant. It provides a detailed analysis of post-route look-up table (LUT4) and register resource utilization, static timing analysis (STA) bottlenecks across the 20

MHz and 50 MHz clock boundaries, formal specification compliance results, and a physical hardware validation sweep utilizing the general-purpose peripherals.

In Chapter 5, the conclusion and recommendations for future work are presented. This chapter summarizes the performance insights established across the three developmental phases and evaluates the project's success against its primary objectives, confirming 100% RISCOF ISA compliance and successful timing closure. Finally, it outlines the architectural limitations of the current implementation and proposes concrete avenues for future research, including the integration of dynamic branch prediction, cache memory hierarchies, and advanced ISA extensions.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter provides a comprehensive literature review and critical analysis of the research landscape surrounding low-power, adaptive RISC-V processor architectures. As demand for edge computing and AIoT applications increases, the limitations of conventional fixed-pipeline designs become a critical focal point for microarchitectural innovation. By evaluating historical perspectives alongside recent technological advancements, this review seeks to identify current research gaps and establish a theoretical foundation for a dynamically scalable 3-to-5 stage pipeline techniques.

To systematically explore the technical requirements of this work, the chapter is organized into five subsequent sections. Section 2.2 examines the RISC-V microarchitectural paradigm, detailing the modularity of the RV32I base specification and discussing how the open-source nature of RISC-V provides the flexibility needed for deep RTL customization in power-constrained edge applications. Section 2.3 analyzes instruction pipelining and hazard management, focusing on fixed pipelines and standard microarchitectural methods for mitigating data and control hazards. This section explicitly highlights the physical power and area overheads imposed by traditional forwarding and stalling logic.

Section 2.4 explores low-power VLSI design and adaptive architectures by examining CMOS dynamic power consumption. This section details Dynamic Frequency Scaling (DFS) and combinational power gating as the theoretical mechanisms required to dynamically reduce switching activity and clock frequency.

Section 2.5 reviews related RISC-V implementations, evaluating existing open-source cores to summarize their architectural strengths while critically exposing their limitations regarding rigid, static pipeline depths and power-inefficient hazard mitigation schemes. Finally, Section 2.6 synthesizes the findings of the review to explicitly identify current research gaps, particularly highlighting the absence of fully hardware-verified dynamically adaptive pipelines and the lack of hardware-validated power profiling in current academic literature.

2.2 The RISC-V Microarchitectural Paradigm

The development of highly optimized, energy-efficient microarchitectures requires direct structural modifications to a processor’s datapath. In traditional proprietary ecosystems, microprocessors are often licensed as “black-box” Intellectual Property (IP) cores. These rigid licensing models legally and technically prohibit hardware designers from altering the internal Register Transfer Level (RTL) logic.

Consequently, the RISC-V Instruction Set Architecture (ISA) serves as the foundation for this research. As a free and open-source ISA designed with a clean-slate philosophy, RISC-V avoids historical architectural baggage and modular partitions features into a frozen base and optional extensions [4]. This ecosystem provides designers with the explicit freedom to implement custom, power-saving microarchitectural interventions at the hardware level while maintaining strict compatibility with standard software compilers.

2.2.1 The RV32I Base Integer Instruction Set

The RV32I base integer instruction set serves as the core foundation of the RISC-V architecture. It provides a minimal yet complete set of only 47 basic

instructions, which is sufficient to support modern operating systems and compiler toolchains [4]. To address specific application demands, designers can integrate standardized extensions, such as the ‘M’ extension for hardware multiplication and division, ‘F/D’ for single/double-precision floating-point operations, or ‘C’ for compressed 16-bit instructions to improve code density [5].

From a hardware design perspective, one of the most hardware-efficient features of RISC-V is the use of fixed-position register specifiers. Unlike other architectures where register fields move based on the instruction types, RISC-V keeps the source register (rs1, rs2) and the destination register (rd) at the same bit positions (bits 15 – 19, 20 – 24, and 7 – 11 respectively) across all formats. The functional purposes of these formats are synthesized in Table 2.2, while their specific bit-field layouts are illustrated in Figure 2.2.

Table 2.1: Meaning and Purpose of R, I, S, B, U, J Types in RISC-V [5].

<i>Types</i>	<i>Meaning</i>	<i>Purpose</i>
R-Type	Register-Register Operations	ALU operations between two registers
I-Type	Immediate or Single-Source Operations	ALU immediate ops, loads, system/CSR, JALR
S-Type	Store Operations	Store word/byte/halfword into memory
B-Type	Branch Operations	Conditional branches
U-Type	Upper Immediate Instructions	Large constant generation, PC-relative addressing
J-Type	Jump Operations	Unconditional jumps

RISC-V Instruction Set

Core Instruction Formats

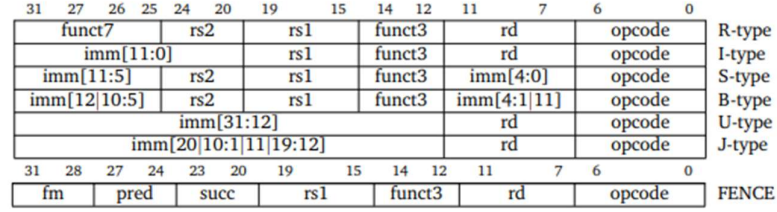


Figure 2.1.: 32-bit RISC-V Instruction Formats (R, I, S, B, U, J format diagram) [5].

This architectural uniformity drastically simplifies the logic gates required in the Instruction Decode (ID) stage. Without multiplexing overhead to extract variable register fields, the decoder's combinational logic depth is significantly reduced. This directly lowers propagation delay and dynamic switching activity which provides the necessary low-power baseline to implement operand isolation.

2.3 Instruction Pipelining and Hazard Management

Instruction pipelining is a fundamental microarchitectural technique that increases instruction throughput by overlapping execution phases. In a single-cycle processor, the clock period is constrained by the total propagation delay of the longest sequential path through all functional units. By inserting pipeline registers to segment large combinational circuits within the datapath, the critical logic path is significantly shortened, allowing for a substantial increase in the maximum operating frequency (f_{max}) [11]. The implementation of such pipelined RISC-V processors on FPGAs has been shown to drastically improve overall throughput compared to their single-cycle or multi-cycle equivalents [3].

2.3.1 Frequency vs. Overhead Trade-Off

Ideally, pipelined architecture aims to achieve a Cycles Per Instruction (CPI) of 1.0. However, the selection of an optimal pipeline depth is a complex optimization problem where frequency gains are constantly balanced against architectural overhead [12].

For ultra-low-power edge applications, the architectural trade-offs between shallower and deeper pipelines are particularly distinct. While a 5-stage pipelined processor effectively decouples memory access from execution to achieve high frequencies, it introduces significant “Pipeline Overhead”. This overhead encompasses the setup times, propagation delays and the increased silicon footprint of the flip-flops introduced at each stage boundary. Research on VexRiscv indicates that deeper pipelines often sacrifice energy efficiency. At subthreshold voltages, such as 300 mV in a 130 nm process, the 2-stage variant outperforms the 5-stage design. The deeper pipeline requires additional buffer logic to maintain cycle times across longer wire routes, inherently increasing dynamic power draw without a proportional reduction in execution time for simple tasks [13].

This severe degradation in energy efficiency shifts the architectural focus toward metrics like the Power-Delay Product (PDP) which is strongly justifying the development of processor cores capable of dynamically reconfiguring their pipeline depths to match real-time workloads and mitigate unnecessary static power dissipation [14].

2.3.2 Data Dependencies and the Hardware Cost of Mitigation

While pipelining enhances throughput, overlapping instructions introduces inherent microarchitectural conflicts known as hazards. Data hazards, specifically

Read-After-Write (RAW) dependencies, occur when an instruction requires the result of a preceding operation that has not yet completed its write-back phase [11].

To prevent the pipeline from computing with stale data, modern architectures implement data forwarding. A Hazard Detection Unit intercepts the required data from intermediate pipeline registers like EX/MEM register or MEM/WB register and routes it directly back to the Execute stage. While forwarding preserves instruction throughput, it needs to pay for heavy physical cost. Integrating multiplexers at the ALU inputs to select between forwarded and register-sourced operands directly increases the routing complexity and the capacitive load of the combinational logic tree [16].

The physical realization of the data forwarding mechanism requires highly synchronized bypass paths. As illustrated in Figure 2.1 which adapted from the [9], the 5-stage pipeline mandates the integration of a dedicated Hazard Unit to manage data dependencies dynamically. The block diagram highlights explicit forwarding paths routed from the Execution (EX), Memory (MEM) and Write-Back (WB) stages directly back to the multiplexer arrays preceding the Arithmetic Logic Unit (ALU). Although the Hazard Unit avoids pipeline stalls through additional routing, this increases layout density and creates critical path bottlenecks, leading to higher capacitive switching power in the Execute (EX) stage.

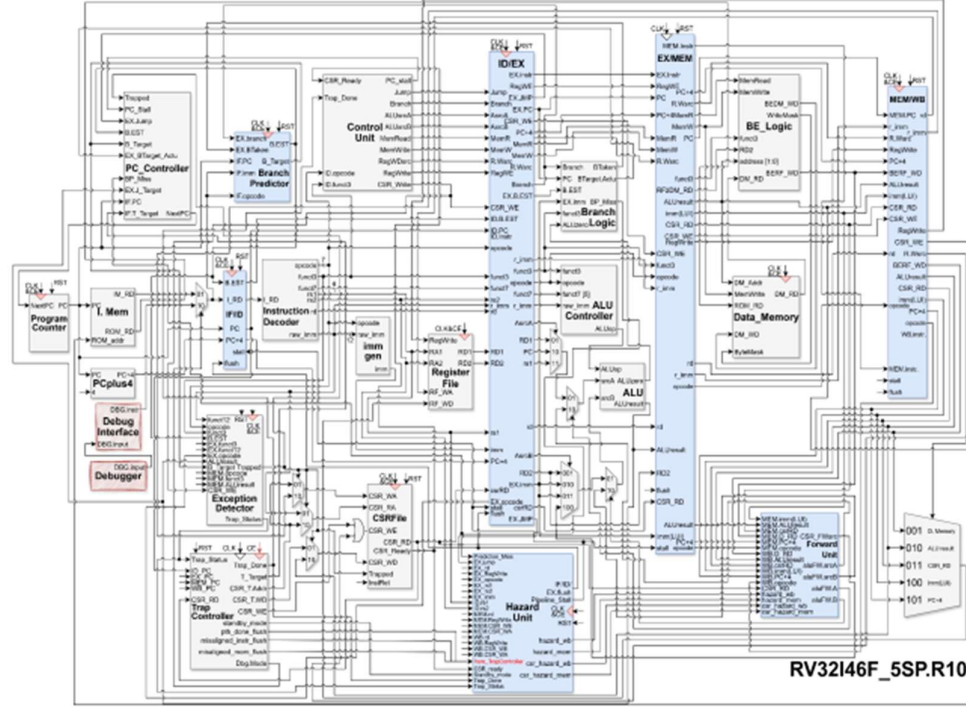


Figure 2.2: Block Diagram of a 5-Stage Pipelined RISC-V Core featuring explicit data forwarding paths and a Hazard Unit. Adapted from [9].

Furthermore, standard forwarding cannot resolve “load-use” dependencies, where data is unavailable until memory access completes. In these instances, the control logic must inject pipeline stalls. Different pipeline designs implement varying levels of hardware overhead to manage data hazards. For instance, standard implementation handles load-use hazards by enforcing a strict, static one-cycle stall in the Instruction Decode (ID) stage [1]. In contrast, researchers have proposed some advanced strategies such as the Stall Scheme Reducing (SSR) architecture which adds dedicated bypass logic between the EX and MEM stages to eliminate these stalls entirely. Implementation data of the SSR scheme within RISC-V pipelines revealed that while performance increased by over 6%, the introduction of complex bypass logic before the execution stage resulted in an increased area footprint and a noticeable surge in dynamic power consumption [8]. Moreover, during a standard stall, the high-

frequency clock continues to toggle the underlying sequential elements, leading to redundant switching activity and inefficient energy use.

2.3.3 Control Hazards and Flush Penalties

Control hazards dictate the most severe pipeline penalties, arising directly from the uncertainty of conditional branch and jump instructions. Due to a pipelined processor fetches subsequent instructions before the branch outcome is resolved in the Execute Stage, it must execute speculatively [11]. If a conditional branch is taken, the speculatively fetched instructions residing in the Fetch and Decode stages become invalid. The processor must flush the pipeline, discard these instructions and waste multiple clock cycles. Crucially, the “flush penalty” also wastes the dynamic energy expended to fetch and decode the discarded instructions that are ultimately discarded [11].

To mitigate these losses, sophisticated branch prediction logic is employed to anticipate control flow outcomes. Figure 2.3 illustrates the architectural implementation of an optimized branch history predictor. By utilizing pattern history tables to track branch behavior, the predictor enables the pipeline to correctly target the Program Counter (PC) before the branch reaches the EX stage, thereby significantly reducing the frequency of costly pipeline flushes. As shown in the figure below, the predictor integrates into the front-end logic to maintain the instruction flow continuity. When a misprediction occurs, the integrated hazard control logic asserts the flush signal, forcing the pipeline registers to clear their contents and insert NOPs while the PC is redirected to the correct target address. This recovery mechanism is vital as the branch prediction balances the necessity of high instruction throughput with the requirement to minimize the energy consumed during recovery phases [7].

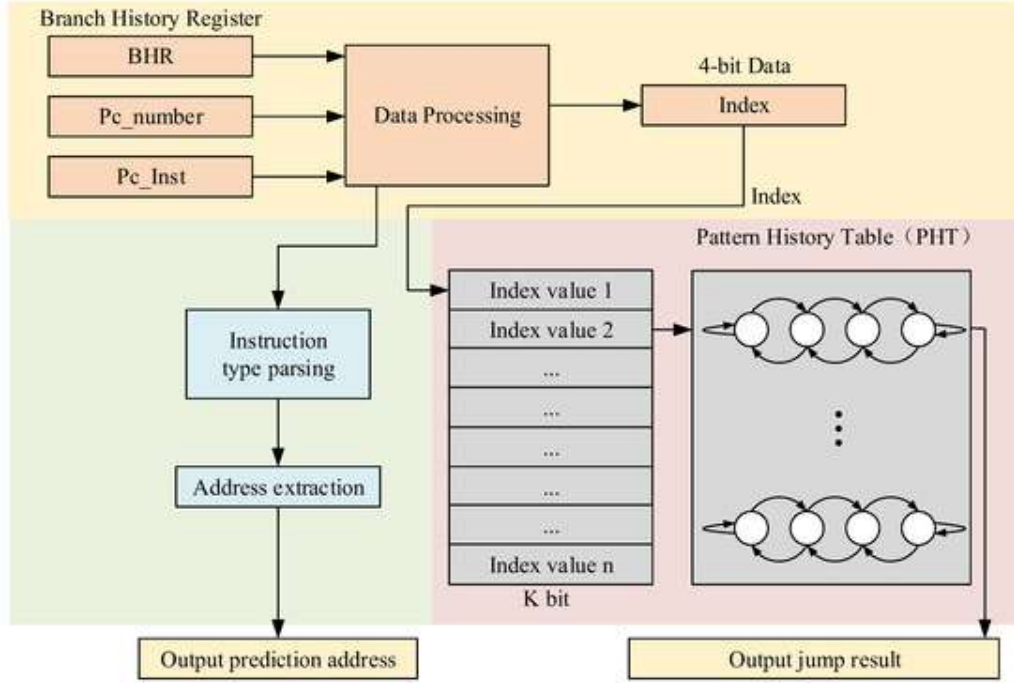


Figure 2.3: Block Diagram of an optimized branch history predictor for RISC-V processors. Adapted from Jin et al. [7].

Different RISC-V cores showcase distinct strategies for surviving control hazards. Deeply pipelined, high-performance superscalar cores adopt complex dynamic branch predictors to avoid massive multicycle flush penalties. However, these structures require significant SRAM and logic overhead, making them hostile to energy-constrained AIoT devices [10]. On the other hand, optimized embedded cores deliberately cap the flush penalty by utilizing shallow pipelines and relying on hardware-efficient static prediction or simplified branch history registers [7]. Some minimalist, size-optimized cores avoid pipeline hazards entirely. For example, the PicoRV32 uses a multi-cycle sequencer instead of a standard pipeline. It eliminates flush penalties but significantly increases the average Cycles Per Instruction (CPI) to approximately 4.0 [19]. Similarly, educational frameworks like MicroRV32 allow designers to evaluate these trade-offs by enabling or disabling specific pipeline features [18].

2.4 Low-Power VLSI Design and Adaptive Architectures

As microarchitectural complexity scales to meet the demands of modern AIoT applications, power consumption has emerged as the primary design constraint. In CMOS-based digital circuits, power dissipation is categorized into static and dynamic components. Although static leakage current is increasingly significant in smaller process nodes, dynamic power, $P_{dynamic}$ continues to dominate in standard pipelined processors, according to

$$P_{dynamic} = \alpha \cdot C \cdot V_{dd}^2 \cdot f \quad (2.1)$$

where α is the switching activity factor, C is the load capacitance, V_{dd} is the power supply voltage, and f is the clock frequency. In a fixed pipeline, α remains significantly high even during stalled cycles [16]. When evaluating a microprocessor, the total power is a summation of the latches ($C_{latches}$) and the functional units (C_{logic}):

$$P_{total} = \alpha \cdot (C_{latches} + C_{logic}) \cdot V_{dd}^2 \cdot f \quad (2.1)$$

Because deepening the pipeline inherently increases the number of sequential latches, it directly inflates the total capacitive load (C_L), leading to higher baseline power consumption regardless of the instruction throughput [12].

2.4.1 Dynamic Frequency and Voltage Scaling (DVFS)

Dynamic Voltage and Frequency Scaling (DVFS) serve as a critical microarchitectural strategy to reduce power dissipation by adjusting operating frequency and supply voltage to meet the performance demands of the current workload, thereby preventing unnecessary energy waste during idle cycles. While simple frequency scaling can reduce activity, it is often insufficient for deep power optimization. To achieve quadratic power savings, processors must also reduce the supply voltage (V_{dd}) as dynamic power is directly proportional to the square of the voltage.

In adaptive RISC-V architectures, Dynamic Voltage and Frequency Scaling (DVFS) is typically implemented via a closed-loop control system that monitors task latency and regulates both the Phase-Locked Loop (PLL) frequency and the Voltage Regulator Module (VRM) output. These designs require a careful trade-off between performance gain and hardware overhead. Static, high-performance cores, such as the standard RISC-V Rocket Core often utilize fixed operating points to simplify timing closure [8]. Conversely, experimental low-power designs often include extra hardware to manage voltage in real-time.

The efficiency between these methods is significant. Clock gating offers a 15-20% reduction in dynamic power [17], while full DVFS can improve battery life up to 40% under varying workloads. However, these gains are partially offset by a larger silicon footprint, as the design must incorporate integrated voltage regulators and synchronization logic to maintain stability during power-state transitions [20].

2.4.2 RTL Level Optimization

To counteract the power overhead introduced by dynamic pipelining, designers employ sophisticated Register Transfer Level (RTL) optimizations. Two of the most prominent techniques include instruction-aware clock gating and adaptive pipeline reconfiguration.

2.4.2(a) Instruction-Aware Clock Gating

A massive portion of dynamic power is consumed by unnecessary switching of combinational logic when functional units are idle. To lower this consumption, architects integrate a Power Management Unit (PMU) to perform clock gating, which effectively shuts off inactive portions of the design such as specific standard cells or entire functional blocks when they are not in used, leaving the core functionality

completely unaffected [17]. In more advanced, complex clock gating schemes can scale power dissipation linearly based on actual port or unit usage, forcing unused units to dissipate only a tiny fraction of their maximum active power [12].

2.4.2(b) Transparent Register Modes and Pipeline Stage Unification (PSU)

Deep pipelines suffer from high power dissipation due to the continuous clocking of intermediate pipeline registers. To reduce the energy per instruction, modern processors utilize Pipeline Stage Unification (PSU), a method that dynamically alters the pipeline depth during program execution by bypassing and inactivating specific pipeline registers [12]. By turning these stage boundaries into transparent paths, adjacent pipeline stages are unified into a shallower pipeline on-the-fly. This theoretical use of transparent register modes avoids redundant stage activity, saving the dynamic power that would have otherwise been consumed by the gated pipeline registers and significantly improving the processor’s overall energy efficiency [12].

2.5 Review of Related RISC-V Implementations

This section evaluates existing RISC-V implementations by focusing on their architectural targets and verification strategies. While many studies successfully address baseline hazard mitigation, there is a significant variance in how these designs are validated for industrial compliance and how efficiently they adapt to varying power constraints.

2.5.1 Thematic Comparison of Core Architectures

Existing research into RISC-V cores typically targets specific static performance or area metrics. Prominent cores such as the Rocket Chip (a highly

parameterized 5-stage, in-order SoC generator) and the MB-Lite (a robust, lightweight 5-stage architecture) implement complex hazard resolution logic but remain fundamentally restricted to fixed pipeline depths [4].

The primary limitation identified in these traditional implementations is the lack of microarchitectural scaling. A predetermined, fixed pipeline depth prevents the core from dynamically optimizing its Power-Delay Product (PDP) and Energy-Delay Product (EDP) when encountering variable workloads or shifting program phases [12]. To understand the microarchitectural choices in modern designs, Table 2.5.1 is used to compare their own architectural characteristics.

Table 2.2: Thematic Comparison of Prominent Cores [8] [12]

<i>Feature</i>	<i>Rocket Core (Standard)</i>	<i>Rocket Core (With SSR)</i>	<i>MB-Lite (RISC-V Adapted)</i>
Pipeline Depth	5-Stage (In-Order)	5-Stage (In-Order)	5-Stage (In-Order)
Hardware Language	Chisel/Scala	Chisel/Verilog	VHDL
Hazard Mitigation	Standard ID-stage hazard detection (1-cycle stall for load-use)	Advanced EXE-to-MEM bypass unit (Zero load-use stalls)	Forwarding operands and stall operation in ID stage
Target Platform	Primarily ASIC	ASIC	FPGA
Key Advantage	High extensibility via RoCC and SoC generators	Maximized IPC by eliminating load-use bubbles	Clean architecture, low resource usage

2.5.2 Verification Methodologies: Testbenches vs. Compliance Frameworks

A critical gap in many academic RISC-V is the reliance on basic, non-automated testbenches for functional verification. Modern Design Verification (DV) standards require more rigorous, standardized approaches to ensure industrial-level reliability.

Traditional functional verification has historically relied on standard Verilog testbenches, utilizing simulation tools such as ModelSim, Iverilog or Verilator to validate a design through directed and randomized stimulus [1] [3] [18]. While these environments are effective for initial debugging and checking basic signal transitions, they are inherently limited by the scope of the testbench’s stimulus generation [18]. In complex pipelined designs, this localized approach often fails to exercise the subtle corner-case hazards, such as specific combinations of data forwarding and branch flushes, that only emerge under exhaustive instruction sequences [18]. Consequently, relying solely on traditional simulation leaves the microarchitecture vulnerable to “silent data corruption” where the core may execute instructions but fail to produce the architecturally correct state in rare pipeline scenarios.

To reach industrial-level reliability, advanced research increasingly adopts RISCOF which is the RISC-V Compatibility Framework and serves it as the industry-standard “Source of Truth” for ISA compliance [3]. Unlike standard testbenches, RISCOF employs a rigorous comparison methodology where the Register Transfer Level (RTL) execution trace of the DUT (Design Under Test) is validated against a “Golden Model”. Typically, this reference is the Spike ISA simulator, a C++ based model specifically developed to serve as the golden standard for RISC-V development [4] [5]. By executing a massive suite of standardized architectural tests, RISCOF ensures that the design adheres strictly to the official ISA specification across all instruction formats and exception handlings [3]. This methodology is particularly vital for the proposed adaptive core as it guarantees that instruction correctness is maintained even as the pipeline dynamically reconfigures its depth and ensures that the hardware remains architecturally solid across both 3-stage and 5-stage operating modes.

2.6 Summary and Identification of Research Gaps

2.6.1 Summary of Findings

The review of the literature demonstrates a profound shift in microprocessor design from proprietary, closed-source architectures to the free, open-source RISC-V Instruction Set Architecture (ISA). This transition has catalyzed immense innovation in custom hardware accelerators and embedded systems by removing licensing barriers [4]. However, across all evaluated architectures, a universal challenge persists which is the strict frequency-power trade-off. While deepening a processor’s pipeline successfully shortens the critical path and increases the maximum operating frequency, it simultaneously introduces severe architectural overheads. In ultra-low power and subthreshold domains, the longer interconnects of deep pipelines demand massive amounts of buffering to meet timing constraints, resulting in drastically increased power consumption without proportional performance gains [13].

To maintain a low Cycles Per Instruction (CPI) in deeper pipelines, researchers have continuously introduced advanced microarchitectural hazard mitigation techniques. Strategies such as data forwarding, dynamic branch prediction, and the Stall Scheme Reducing (SSR) architecture successfully minimize pipeline “bubbles” and improve raw instruction throughput. However, this literature review reveals that these throughput gains uniformly come at the cost of increased silicon area and static power. For example, while the SSR architecture improves Dhrystone throughput by 6.9% by eliminating load-use stalls, it actively increases dynamic power consumption by 8.4% and core area by 6.4% [8]. Similarly, integrating a Simplified Local Branch History Predictor (SLBHP) improves prediction accuracy by 12% but

inflates logic utilization by 24% [7]. Ultimately, static hardware additions designed to mitigate pipeline hazards inherently degrade the Energy-Delay Product (EDP).

2.6.2 Strengths and Limitations of Existing Research

To clarify the current landscape of RISC-V implementations for edge devices, Table 2.6.1 summarizes the strengths and critical limitations of the prevailing architectural approaches identified in this review.

Table 2.3: Strengths and Limitations of Existing Research

<i>Architectural Approach</i>	<i>Core Examples</i>	<i>Primary Strengths</i>	<i>Critical Limitations</i>
Shallow/Multi-Cycle Pipelines	Vex-2 PicoRV32 Zero-riscy	Extremely low area footprint and highly energy-efficient in subthreshold environments.	ALU and memory access occur in the same cycle, creating severe frequency bottlenecks and low absolute throughput.
Standard Fixed 5-Stage Pipelines	MB-Lite Standard Rocket	Effectively decouples memory and execution. Standard and robust compilation flow.	Rigid depth incurs high buffer overhead and redundant register clocking during simple arithmetic phases.
Aggressive Hazard Mitigation	Rocket-SSR Rev-riscv	Maximizes IPC by bypassing load-use stalls and highly accurate branch flow.	Heavy multiplexer and LUT overhead result in dynamic power consumption increases, making them hostile to ultra-low-power constraints.
Traditional Functional Verification	ModelSim Verilator	Excellent for localized debugging and verifying basic module transitions.	Fails to simulate complex, exhaustive corner-case pipeline hazards and susceptible to silent data corruption.

2.6.3 Identification Research Gaps

Despite extensive research into RISC-V microarchitectures, current solutions fail to adequately balance the high throughput of a 5-stage pipeline with the energy

efficiency of a 3-stage pipeline under fluctuating workloads. This literature review identifies three primary research gaps.

2.6.3(a) Rigidity in Pipeline Depth

The most significant limitation of existing open-source RISC-V cores (such as Rocket, MB-Lite, and VexRiscv) is their static nature as their pipeline depth is permanently fixed at the time of hardware generation [4]. Literature suggests that dynamically altering pipeline depth such as through Pipeline Stage Unification (PSU) could drastically reduce power consumption by jumping over redundant registers during simple instruction phases [12]. However, practical, fully verified implementations of adaptive 3-to-5 stage scaling in open-source RISC-V cores remain largely unexplored.

2.6.3(b) Penalty of Static Hazard Mitigation

While research into the SSR microarchitecture and advanced branch predictors successfully address data and control hazards, these solutions impose permanent power and area penalties [7]. There is a gap in research regarding microarchitectures that can dynamically remove the source of the hazard by unifying the pipeline stages themselves rather than merely adding more bypass logic to a fixed pipeline.

2.6.3(c) Lack of Industrial-Grade Verification for Adaptive Cores

Most academic proposals for novel pipelining or hazard mitigation rely on standard, non-automated HDL testbenches. There is a distinct lack of research that applies rigorous, industrial-grade Design Verification (DV) specifically the RISC-V Compliance Test Framework (RISCOF) and Spike Golden Model to guarantee that dynamically reconfigurable or adaptive pipelines maintain 100% architectural correctness across changing states [3] [4]. This gap is especially critical for a core that

must seamlessly switch between 3-stage and 5-stage modes, as transition states introduce novel corner-case hazard combinations that cannot be adequately exercised by directed testbenches alone.

2.6.4 Project Justification

By developing a selectively adaptive pipeline that bridges the gap between shallow energy efficiency and deep execution throughput, accompanied by RISCOF-driven verification, this project aims to provide a highly reliable, power-scalable solution for next-generation embedded systems.

CHAPTER 3

METHODOLOGY

3.1 Introduction and Project Workflow

This chapter outlines the systematic methodology adopted to design, verify, and evaluate an adaptive, low-power RV32I processor core. The primary research goal is to develop a microarchitecture capable of dynamically scaling its pipeline depth between 3 and 5 stages to achieve a 50% reduction in dynamic power consumption while maintaining 100% Instruction Set Architecture (ISA) compliance. By transitioning from a validated single-cycle baseline, through a static pipelined architecture, and ultimately into an adaptive datapath, this methodology provides a practical engineering framework for optimizing energy efficiency in IoT edge-class devices.

To ensure a structured and reliable development cycle, the methodology follows an iterative digital VLSI design flow. As illustrated in Figure 3.1 below, the process is segmented into three primary developmental phases which are Baseline Validation (Phase 1), Pipelined Architecture (Phase 2), and Adaptive Architecture (Phase 3). To ensure design consistency, each development phase requires passing unit-level tests and system-level compliance checks before moving to FPGA synthesis.

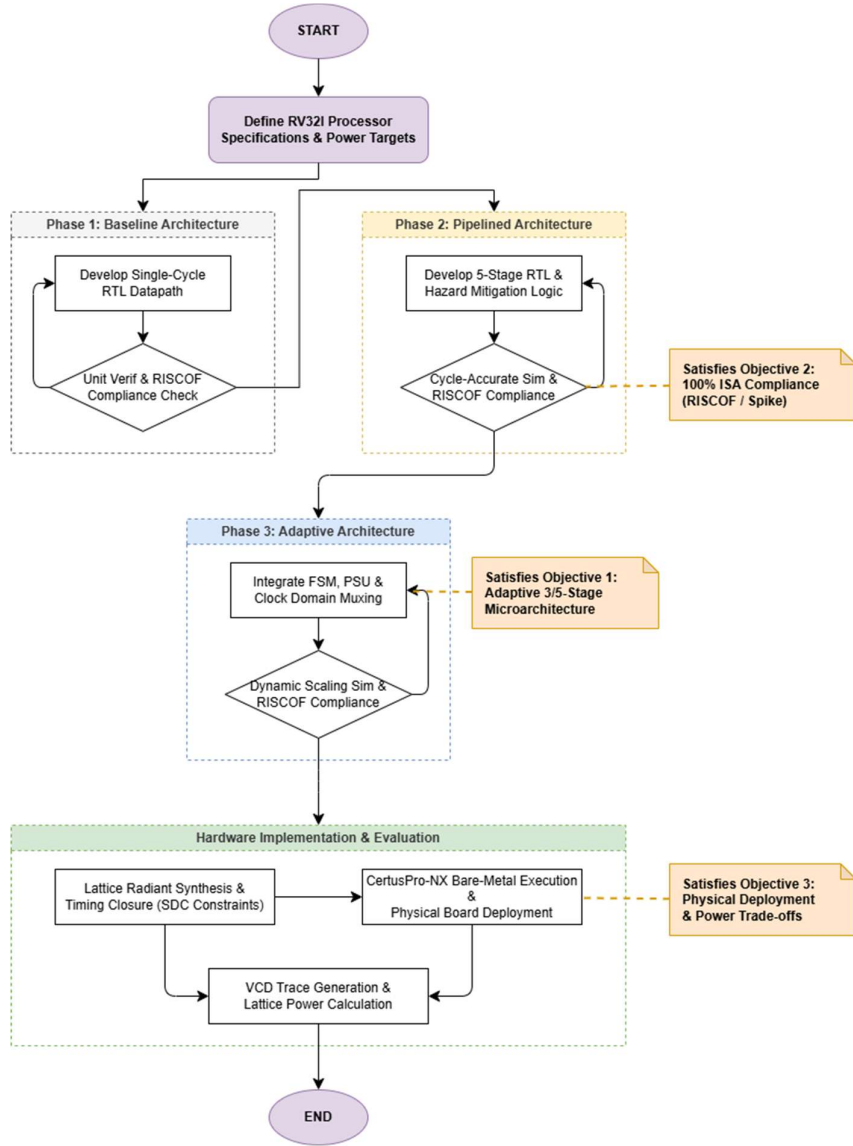


Figure 3.1: Three-Phase Iterative Design Flow

To provide a logical progression from toolchain setup to architectural deployment, the remainder of this chapter is organized into the following sequential sections. Section 3.2 details the development environment and hardware specifications, specifically highlighting the hybrid software toolchain and the 28nm FD-SOI Lattice CertusPro-NX FPGA platform utilized for hardware synthesis and post-route power analysis. Following this, Section 3.3 explains the Phase 1 baseline single-cycle architecture, covering the foundational datapath, control logic, and

Harvard memory interfaces that serve as the golden reference for the design. Section 3.4 then details the Phase 2 transition to a fixed 5-stage pipeline through the structural segmentation of the datapath and the insertion of critical hazard mitigation mechanisms, including data forwarding and load-use stalling logic.

Section 3.5 explores Phase 3, introducing the adaptive pipeline architecture by explaining the integration of Pipeline Stage Unification (PSU), glitch-free clock domain multiplexing, and combinational power gating to achieve runtime structural elasticity. Section 3.6 outlines the system verification and synthesis strategy, presenting a multi-tiered verification plan focused on architectural compliance testing using the RISCOF framework and the Spike golden reference model, alongside the physical synthesis constraints required for accurate power profiling. Finally, Section 3.7 summarizes the methodological progression and establishes the foundation for the empirical hardware results evaluated in the subsequent chapter.

3.2 Development and Hardware Specification

To verify that the simulated architectural translates to physical hardware, the design must be synthesized and profiled on an appropriate platform. This section details the hardware and software development environment used to build and evaluate the adaptive RISC-V core.

3.2.1 Hardware Prototyping Platform

The physical prototyping and power characterization of the design are conducted on the Lattice Certus-NX Pro Versa Board. This FPGA board was selected because it is specifically optimized for low-power edge applications, providing the necessary hardware resources to evaluate microarchitectural scaling.

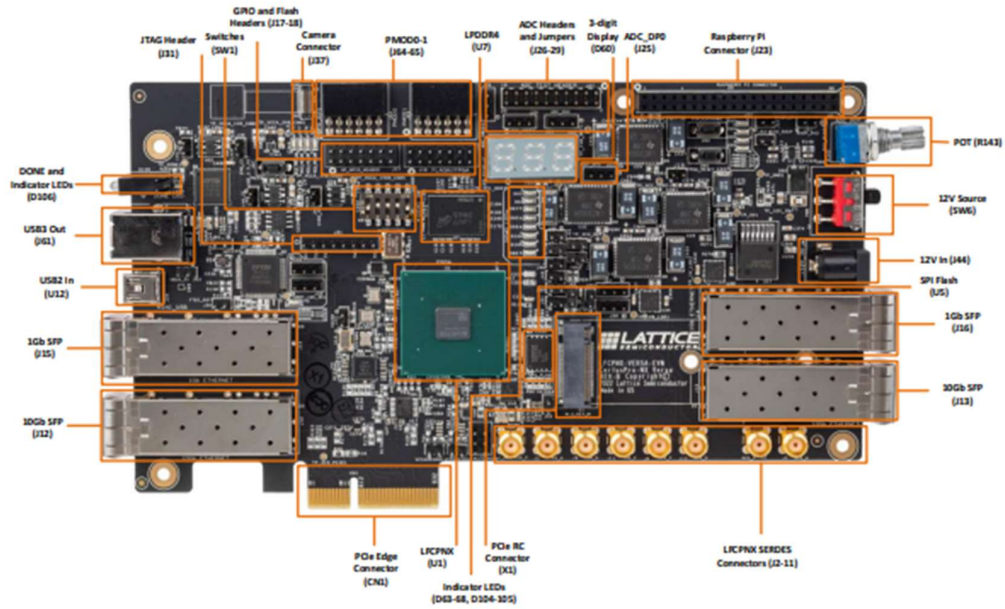


Figure 3.2: Top View of CertusPro-NX Versa Evaluation Board

The board features the LIFCL-40 FPGA, which is built on the Lattice Nexus platform utilizing a 28nm Fully Depleted Silicon-On-Insulator (FD-SOI) process. Unlike traditional bulk CMOS FPGAs, the FD-SOI technology provides a significantly lower “floor” for static power leakage and allows for superior performance-per-watt at reduced operating voltages.

In terms of computational capacity, the LIFCL-40 device offers approximately 39,000 Look-Up Tables (LUTs) and 2.9MB of Embedded Block RAM (EBR). These resources are more than sufficient to implement the full 5-stage RV32I datapath, the adaptive stage-unification logic, and the associated forwarding networks. To facilitate accurate performance evaluation, the Versa Board is equipped with dedicated power rails and low-side sensing resistors for the V_{CC} (Core) and V_{CCIO} (I/O) supplies. Within the project methodology, these hardware sensors are utilized in conjunction with the Lattice Radiant Power Calculator to extract high-fidelity switching activity data. This

ensures that the evaluation of dynamic power (P_{dyn}) is based on physical mapping and actual signal toggling rates rather than purely theoretical simulation.

3.2.2 Hybrid Development Environment (WSL2)

The project utilizes a hybrid development environment consisting of Windows 11 and the Windows Subsystem for Linux (WSL2) to bridge the gap between proprietary FPGA tools and open-source verification frameworks. This setup is specifically configured to support the RISC-V GNU Toolchain, which is required to compile high-level C-based codes into RV32I machine code.

The justification for this hybrid architecture is rooted in toolchain compatibility; the RISCOF framework and Spike simulator are optimized for Unix-based environments, and WSL2 provides a native Linux kernel that allows these tools to run at peak performance without environmental conflicts. This ensures fast iteration during the functional verification of the exhaustive compliance suites.

3.2.3 Synthesis and Simulation Toolchain

At the foundation of the design phase, the processor is modeled at the Register-Transfer Level (RTL) using Verilog-2001. Verilog provides the necessary modularity to explicitly define hardware behavior and clock-domain boundaries, which is essential for synthesizing the adaptive multi-stage pipeline.

To ensure functional correctness before physical FPGA deployment, cycle-accurate simulations are performed using QuestaSim and Verilator. These simulation environments provide deep, cycle-by-cycle signal visibility, allowing for the rapid identification and debugging of transient hazards that could easily be overlooked during static code analysis.

Table 3.1: Summary of Project Development and Verification Tools

<i>Category</i>	<i>Tool</i>	<i>Purpose</i>
HDL	Verilog-2001	RTL modeling of the 3-to-5 stage core.
Simulation	Questasim / Verilator	Functional verification and .vcd files generation.
Synthesis	Lattice Radiant (LSE)	Logic mapping and timing closure for CPNX-Versa FPGA board.
Compliance	RISCOF + Spike	“Golden Model” verification for ISA compliance.
Toolchain	RISC-V GNU Toolchain	Compiling C codes to machine codes.

3.3 Phase 1: Baseline Single-Cycle Architecture

The initial milestone of the methodology involved the design and unit-level verification of a single-cycle RV32I processor. This unpipelined core serves as the foundation for the project. By forcing every instruction to complete its entire Fetch-to-Write-Back sequency synchronously within a single clock period, this phase ensures that the core instruction logic, control signaling, and datapath steering are architecturally sound before the complexity of multi-stage pipelining is introduced.

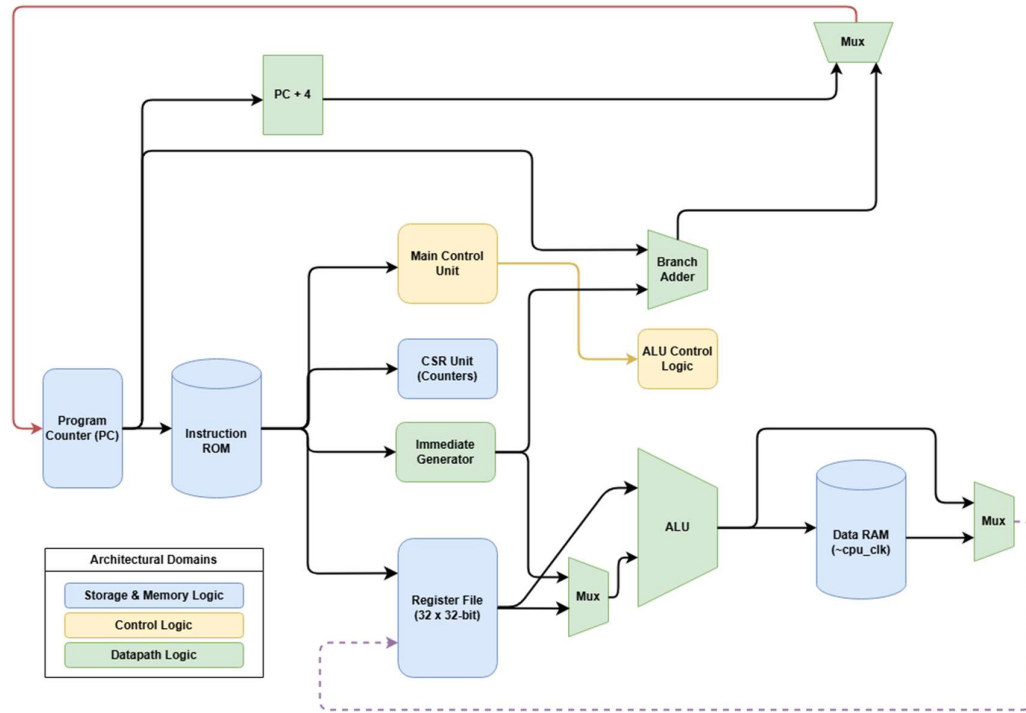


Figure 3.3: Microarchitecture of the Baseline Single-Cycle RV32I SoC

As illustrated in Figure 3.3, the baseline single-cycle core integrates with 128 KB Instruction ROM and a 128 KB Data RAM to form a self-sufficient computing system. To provide a clear conceptual bridge toward the future multi-stage adaptive pipeline, the architecture is explicitly decamped into three primary logic domains, distinguished by color in the diagram which are Control Logic as Yellow, Datapath Logic as Green, and Storage & Memory Logic as Blue.

In this unpipelined configuration, every instruction completes the entire data flow synchronously. Each domain consists of specific Verilog modules that collaborate to stabilize all combinational signals before the next rising clock edge triggers the final write-back phase.

3.3.1 Control Logic Domain

The control logic serves as the “brain” of the processor, responsible for interpreting machine code and steering the flow of data. In this implementation, the decoding is parallelized to minimize the combinational delay.

3.3.1(a) Main Control Unit

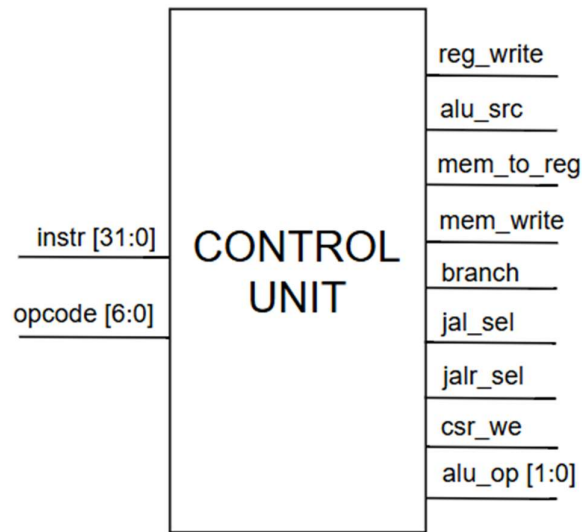


Figure 3.4: I/O Ports for Control Unit Module

The control unit module serves as the core decoder, receiving the 7-bit opcode directly from the instruction stream. It is using these 7-bits opcodes to determine the instruction type such as R-Type, I-Type or B-Type to produce high-level control signals that define the flow of data across the entire processor. Key outputs include `alu\_src`, which determines whether the ALU operates on a second register operand or an immediate value, and `reg\_write`, which enables the synchronous update of the Register File. Additionally, it manages memory interactions through the `mem\_write` signal to trigger Data RAM stores and the `mem\_to\_reg` signal to select

between computational results or memory-retrieved data for the final Write-Back (WB) stage.

3.3.1(b) ALU Control Logic

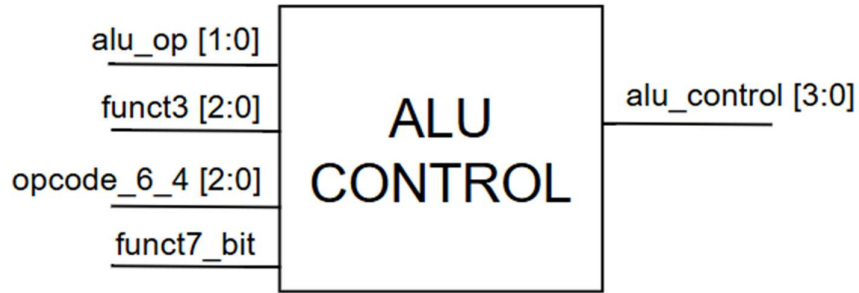


Figure 3.5: I/O Ports for ALU Control Module

To prevent the main controller from becoming a critical timing bottleneck, the design offloads specific operational decoding to the ALU Control Module. This unit refines the 2-bit 'alu_op' signal received from the main controller by analyzing the 'funct3' and 'funct7' instruction fields. By focusing on these sub-fields, the ALU Control Unit generates a precise 5-bit 'alu_control' signal. This high-resolution command from the ALU Control Unit allows the ALU Module to distinguish between operations that share similar opcodes but required different functional logic, such as differentiating between an addition and a subtraction, or an arithmetic right shift (SRA) and a logical right shift (SRL). This hierarchical decoding strategy ensures that the processor maintains 100% architectural correctness while meeting the target operating frequency on the Lattice FPGA board.

3.3.2 Datapath Logic Domain

The datapath logic domain encompasses the functional units responsible for the data manipulation and transformation as it flows through the processor. In this

baseline single-cycle implementation, these modules are designed as large combinational blocks to ensure that all arithmetic and logical transitions stabilize within the primary clock period.

3.3.2(a) Arithmetic Logic Unit (ALU)



Figure 3.6: I/O Ports for ALU Module

The ALU Module serves as the central computational engine of the processor, executing a wide array of 32-bit operations directed by a 5-bit 'alu_control' signal. To optimize timing and reduce logic levels, the module precomputes common operations such as the 32-bit 'sum' and 'diff' before the final output selection. Its computational features include standard arithmetic (ADD, SUB), bitwise logical operations (AND, OR, XOR) and a variety of shift functions including Logical Left Shift (SLL), Logical Right Shift (SRL), and Arithmetic Right Shift (SRA). Beyond data transformation, the ALU plays a critical role in control flow by precomputing a 'zero' flag and a 'less_than' signal. These signals are utilized by the branch logic to evaluate conditional jumps, such as BEQ or BLT, ensuring the processor correctly navigates the program's control structures.

3.3.2(b) Immediate Generator

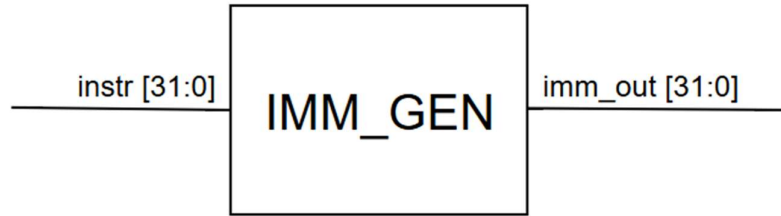


Figure 3.7: I/O Ports for Immediate Generator Module

The Immediate Generator Module is a dedicated hardware module designed to extract, and sign-extend immediate values encoded within the instruction stream. Because the RISC-V ISA utilizes various bit-field layouts to maintain fixed register positions, the Immediate Generator Module must dynamically reassemble fragments from the 32-bit instruction depending on the opcode. This implementation provides comprehensive support for I, S, B, U and J-Type formats, accurately reconstructing signed 32-bit operands for operations like `'addi'`, memory offsets for load/store instructions, and PC-relative offsets for jumps and branches.

3.3.2(c) Program Counter (PC)

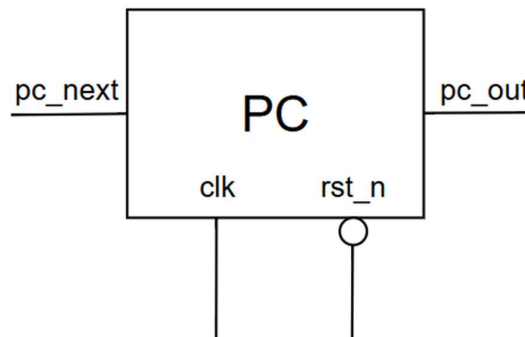


Figure 3.8: I/O Ports for PC Module

The Program Counter Module stands as the singular sequential element within the otherwise combinational core datapath. It is responsible for maintaining the current

instruction address and driving it into the instruction memory, ROM. On every rising edge of the system clock, `cpu\_clk`, the PC updates its internal register to the value of `pc\_next`, which is determined by the control logic as either the sequential address (PC + 4) or a target jump/branch address. To ensure a deterministic boot sequence, the PC is equipped with an active-low reset that initializes the pointer to `0x8000\_0000`, successfully aligning the processor with the standard RISC-V memory map required for the execution of official architectural compliance tests.

3.3.3 Storage and Memory Logic Domain

The storage and memory logic domain is responsible for the persistent state of the processor and the retrieval of data necessary for computation. This domain ensures that the results generated by the datapath are stored correctly and that the instruction stream remains uninterrupted.

3.3.3(a) Register File

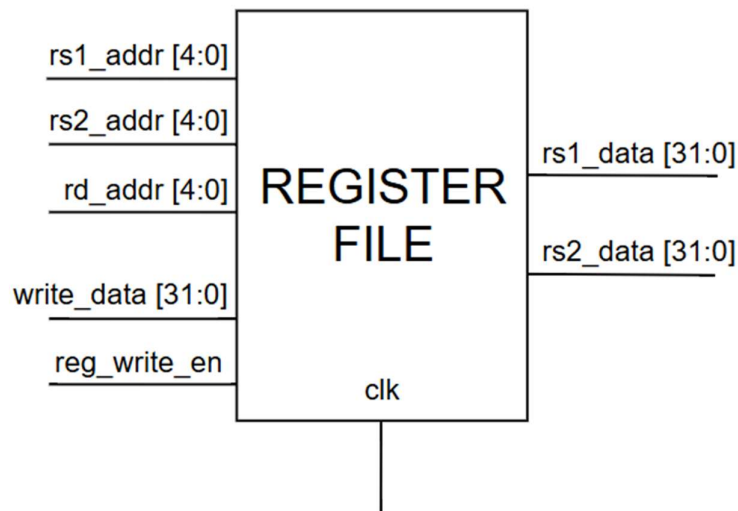


Figure 3.9: I/O Ports for Register File Module

The Register File Module serves as the primary high-speed and temporary storage for the processor, implementing 32 general-purpose registers (`x0` to `x31`), each 32 bits wide. To meet the timing requirements of single-cycle architecture, the module is designed with asynchronous read capabilities, providing two dedicated read ports that allows `rs1\_data` and `rs2\_data` to be accessed combinationaly when the instruction addresses are decoded. Conversely, the module utilizes synchronous write logic to ensure data stability which writing to the destination register, `rd` occurs exclusively on the positive edge of the system clock. This write operation is gated by the `reg\_write\_en` signal from the Control Unit and includes a hardware check to prevent modification of the `x0` register, which is hard-wired to zero in RISC-V specification.

Registers

General Purpose Registers

Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller

Figure 3.10: General Purpose Registers in RISC-V Specifications

3.3.3(b) Control and Status Register (CSR) Unit

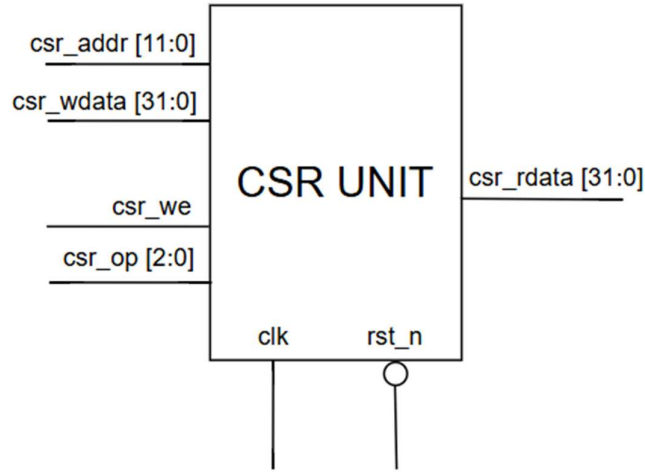


Figure 3.11: I/O Ports for CSR Unit Module

The Control and Status Register (CSR) Unit is integrated into the system to provide critical performance monitoring and system-level status tracking. The unit manages 640bit performance counters, specifically 'mcycle' and 'minstret', which track the total elapsed clock cycles and the number of instructions successfully retired respectively. These counters are essential for the methodology's evaluation phase, as they provide the empirical data needed to calculate the Cycles Per Instruction (CPI) and verify performance during industry-standard benchmarks like CoreMark and Dhrystone. Furthermore, the CSR unit also acts as a compliance interface, allowing the processor to interact with the RISCOF framework to signal internal tests during architectural testing.

3.3.3(c) Instruction and Data Memory Interface

To maximize throughput and prevent resource contention, the processor utilizes Harvard architecture, which provides independent pathways for instruction and data access.

Instruction ROM

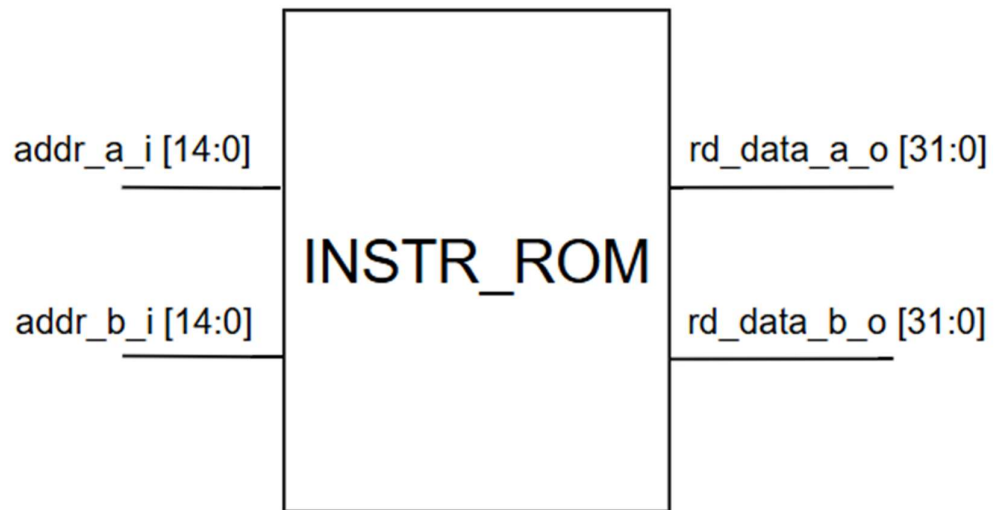


Figure 3.12: I/O Ports for Instruction ROM Module

The instruction memory is implemented as a custom-designed Dual-Port ROM module. Port A is dedicated to the Instruction Fetch (IF) stage via the Program Counter module, while Port B provides the ALU with a secondary read path for accessing constants or strings stored within the code segment.

Data RAM

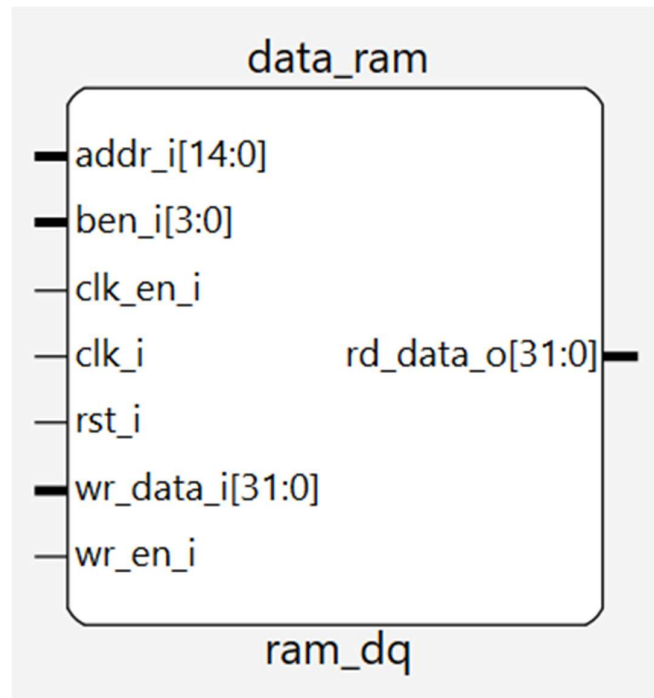


Figure 3.13: I/O Ports for Data RAM Module

In contrast, Data RAM utilizes a Lattice ``ram_dq`` IP block, which is mapped directly to the FPGA's Embedded Block RAM (EBR) primitives. By leveraging the native Lattice IP, the design achieves significantly higher resource efficiency while minimizing the use of precious LUT4 logic.

To maximize the timing slack within the single-cycle window, the Data RAM is driven by an inverted clock, ``~cpu_clk``. This VLSI optimization provides the ALU results and address decoding logic about half-cycle window to stabilize before the RAM captures the data at the mid-point of the CPU clock cycle, effectively eliminating timing violations and “silent data corruption” at higher operating frequencies.

3.4 Phase 2: Transition to the Fixed 5-Stage Pipeline

While the single-cycle baseline proved the functional correctness of the instruction decoding and datapath logic, its long combinational critical path strictly limited the maximum operating frequency ($F_{\max}$). To meet the project's high-performance target of 50 MHz, Phase 2 of the methodology transitions the core into a statically pipelined microarchitecture. By segmenting the large combinational logic chain into smaller, concurrent execution stages, the propagation delay per clock cycle is minimized, significantly increasing the instruction throughput.

3.4.1 Pipelining the Datapath (Structural Segmentation)

The single-cycle datapath was systematically sliced into the classic five-stage RISC-V execution model which are Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Write-Back (WB).

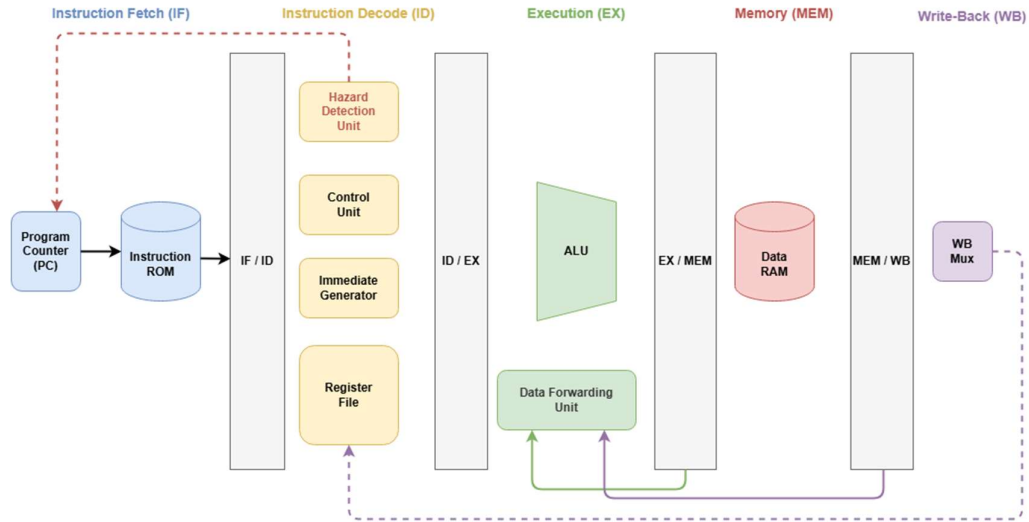


Figure 3.14: Structural Segmentation of the 5-Stage RV32I Pipelined Datapath

To isolate these distinct logic domains, synchronous pipeline registers were implemented at the boundaries between each stage. On every positive clock edge, the instruction state, operands, and necessary control signals propagate to the next stage. This structural transition allows the processor to process up to five separate instructions simultaneously. However, running multiple instruction concurrently introduces pipeline hazards that require hardware mitigation logic.

3.4.2 Hazard Mitigation Strategies

The core challenge of pipeline design is resolving the data and control hazards that occur when an instruction requires a resource or a computational result that is currently locked by a preceding instruction. To maintain 100% ISA compliance without artificially stalling the entire pipeline, three dedicated hazard resolution modules were introduced.

1. Data Forwarding Unit (Resolving RAW Hazards)

Read-After-Write (RAW) data hazards occur when an instruction in the ID stage requires an operand that is still being calculated by an older instruction in the EX, MEM, or WB stages. As illustrated in the pipeline timing diagram (Figure 3.15),

a dependent instruction would be forced to wait for the Register File to update causing severe pipeline stalls if without any intervention.

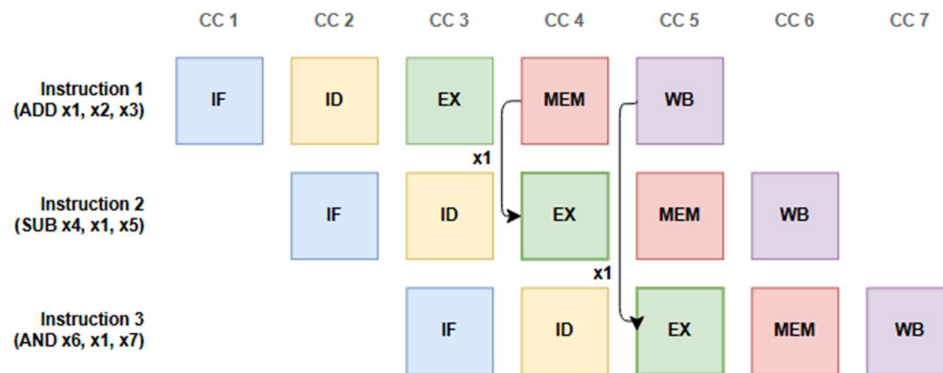


Figure 3.15: Read-After-Write (RAW) Data Hazard

To prevent these stalls and maintain high instruction throughput, a dedicated Forwarding Unit module was implemented. This combinational module continuously monitors the destination registers of the instructions currently locked in the EX.MEM and MEM/WB pipeline registers. If a match is detected against the source registers of the instruction entering the EX stage, the Forwarding Unit dynamically passes the required data backward through time directly into the ALU input multiplexers so that the data able to bypass the Register File entirely.

2. Hazard Detection Unit (Resolving Load-Use Stalls)

While the Forwarding Unit successfully resolves arithmetic RAW hazards, it cannot resolve “load-use” hazards. This specific temporal hazard occurs when an instruction attempts to use a register immediately after a memory load targets that same register. Because the required data is not physically extracted from the Data RAM until the MEM stage, it is impossible to forward it in time for the dependent instruction to use it in the EX stage.

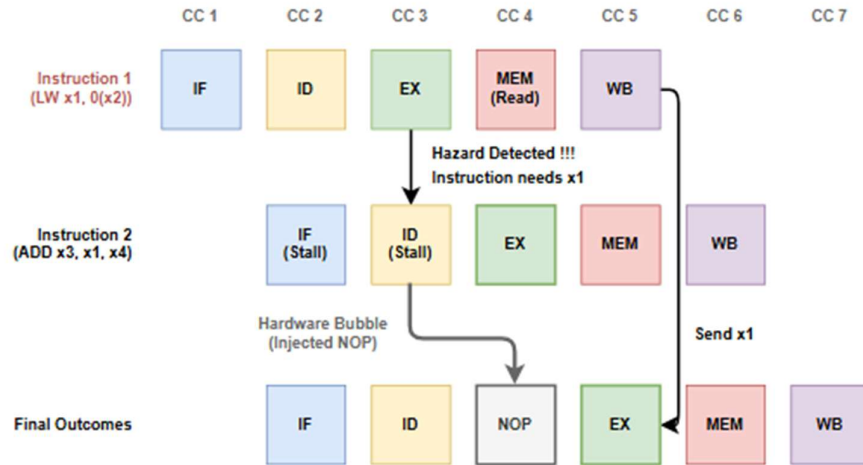


Figure 3.16: Load-Use Stall Hazard

To resolve this hardware limitation, a Hazard Detection Unit was integrated into the ID stage. This module monitors the control signals of the instruction currently in the EX stage. If the EX instruction is a memory load, and its destination register matches source register of the decoded instruction in the ID stage, the unit asserts a stall signal. This signal physically locks the states of the Program Counter and IF/ID pipeline register. Simultaneously, it effectively injects a hardware pipeline bubble into the execution stream by forcing the ID/EX register to load ‘NOP’ instruction to delay the dependent instruction by exactly one clock cycle until the memory data becomes available for standard forwarding.

3. Branch Control Hazard Resolution (Pipeline Flushing)

Control hazard inherently arises from branch instructions. Because the branch condition and target address are not resolved by the ALU until the EX stage, the processor speculatively fetches the two subsequent sequential instructions into the IF and ID stages. As illustrated in the pipeline timing diagram (Figure 3.17), if the branch evaluates as “taken”, this speculative execution path becomes invalid and must be immediately discarded.

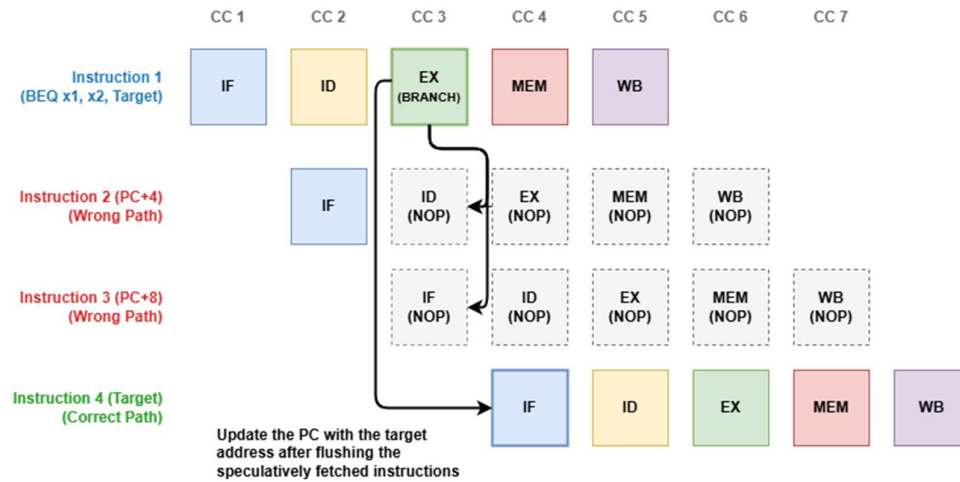


Figure 3.17: Control Hazard with Pipeline Flushing

To preserve architectural correctness, a hardware flush mechanism was implemented within the datapath. Upon detecting a taken branch in the EX stage as (Clock Cycle 3 in Figure 3.17), the control logic asserts a 'flush' signal. On the subsequent clock edge, this signal synchronously forces both the 'reg_if_id' and 'reg_id_ex' pipeline registers to clear their contents by replacing with standard RISC-V NOP bubbles (32'h0000_0013). Simultaneously, the Program Counter latches the calculated target address, enabling the IF stage to fetch the correct instruction stream by Clock Cycle 4 without corrupting the processor's state.

3.5 Phase 3: The Adaptive Architecture

While the transition to a fixed 5-stage pipeline core was necessary to reduce combinational delay and elevate the maximum operating frequency, pipelined architectures inherently suffer from increased dynamic power consumption. The continuous toggling of intermediate pipeline registers wastes energy while executing low complexity instructions. To address this and fulfill the primary research objective of minimizing dynamic power, Phase 3 evolves the core into an adaptive, software-

reconfigurable microarchitecture. This advanced design is capable of dynamically collapsing from a deep 5-stage pipeline into a shallow, energy-efficient 3-stage pipeline at runtime.

3.5.1 Pipeline Stage Unification (PSU)

The core mechanism enabling this structural elasticity is Pipeline Stage Unification (PSU). Initially, the theoretical model proposed a pure 3-stage unification which collapsed EX, MEM, and WB into a single cycle. However, Static Timing Analysis (STA) revealed that concatenating the ALU propagation delay, Data RAM access time, and final Write-Back multiplexing into a single unregistered combinational path created severe setup timing violations caused the targeted 25 MHz clock frequency failed to meet.

To resolve this critical path failure and achieve timing closure, the architecture was modified to implement a pseudo-4th-stage barrier and dynamic multiplexing, so that it able to safely collapse the execution flow. The complete adaptive microarchitecture is illustrated in Figure 3.18.

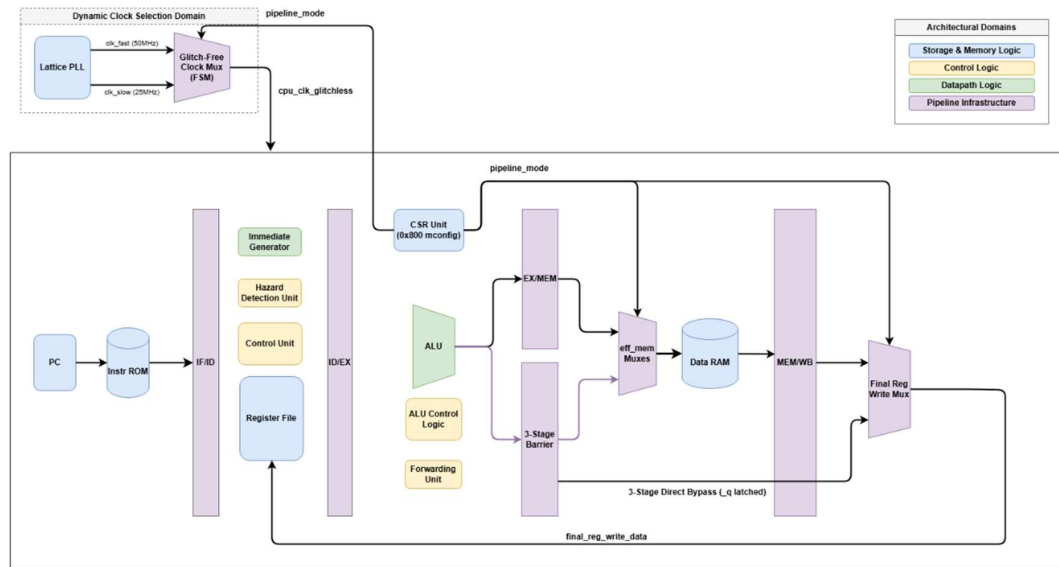


Figure 3.18: Microarchitecture of the Adaptive Core

Writing to the custom Control and Status Register (``mconfig`` at address 0x800) drives the ``pipeline_mode`` signal across the datapath. To visualize how the processor physically achieves structural elasticity, Figure 3.19 demonstrates the core actively operating in the energy-efficient 3-stage mode (``pipeline_mode`` = 0).

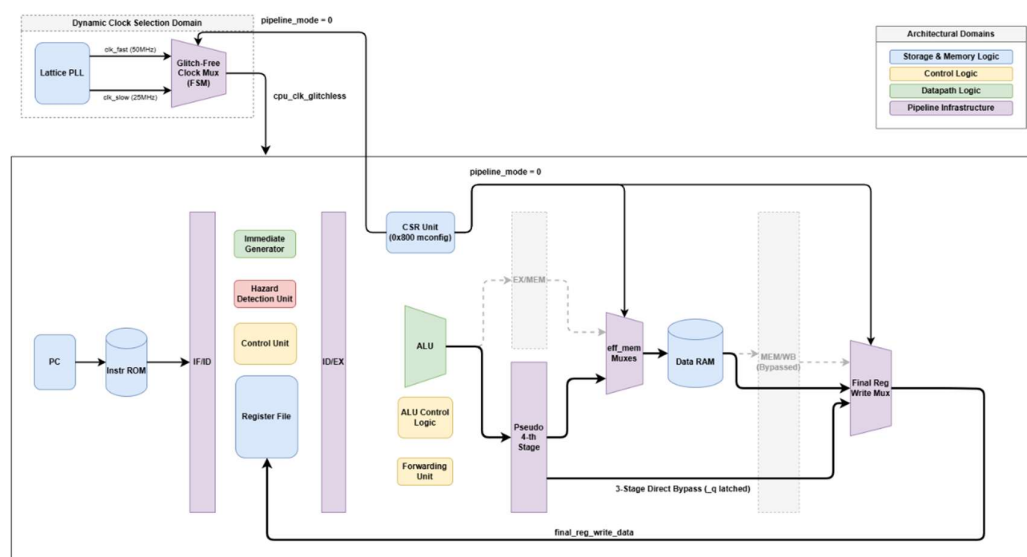


Figure 3.19: Adaptive Core operating in 3-Stage Mode

Upon transitioning to the low-power 3-stage mode, the processor dynamically reconfigures its datapath routing through three primary mechanisms. First, a pseudo-4th-stage barrier is integrated to bypass the standard EX/MEM and MEM/WB pipeline registers. Rather than propagating through the full 5-stage chain, the ALU execution results, evaluated branch conditions, and immediate values are latched into a dedicated set of Write-Buffer Barrier registers at the EX stage boundary. Although this design necessitates a physical 4-stage register structure to ensure timing closure at the required operating frequency, the architecture maintains the functional behavior of a 3-stage pipeline by unifying the Execution (EX), Memory Access (MEM), and Write-Back (WB) stages into a single logical block.

To facilitate this unification, the design employs adaptive memory routing. Multiplexers positioned before the Data RAM select the output from the pseudo-4th-stage barrier instead of the EX./MEM boundary utilized in 4-stage mode, thereby routing execution results directly into the memory access logic. Finally, the Final Write-Back Multiplexer selectively bypasses the MEM/WB pipeline register, allowing the latched barrier data or the Data RAM output to route directly back to the Register File within a single clock cycle.

By making these boundaries transparent via hardware multiplexing, the Execution (EX), Memory Access (MEM), and Write-Back (WB) stages are unified into a single, extended logical block. This allows the architecture to functionally operate as a highly efficient 3-stage pipeline (IF -> ID -> EX/MEM/WB), minimizing dynamic register toggling while preserving safe combinational path lengths.

3.5.2 Glitch-Free Dual-Clock Domain Architecture

Collapsing three pipelined stages into a single combinational block drastically increases the propagation delay path. Consequently, the unified low-power mode cannot safely operate at the 50 MHz frequency used by the 5-stage mode without inducing severe timing violations. However, this frequency reduction is not merely to meet Static Timing Analysis (STA) constraints, it actively serves the core research objective. Because dynamic power consumption in CMOS logic is directly proportional to the operating frequency, reducing the clock speed to 20 MHz inherently reduces the baseline switching power by 60%. Therefore, to ensure both architectural stability and maximized energy efficiency, the processor implements a

dual-domain clocking strategy which a fast clock (50 MHz) for high-performance 5-stage execution and a slow clock (20 MHz) for the adaptive 3-stage execution.

However, dynamically switching between two asynchronous clock domains using simple combinational logic gates creates clock glitches. If a clock edge is cut short during a transition, it violates the setup and hold times of the internal flip-flops, leading to catastrophic metastability and silent data corruption within the processor state.

To safely manage this constraint and enforce a strict hardware transition protocol, a dedicated Mode Switch Controller was implemented. As detailed in Figure 3.19, this controller utilizes a 4-state Finite State Machine (FSM) to actively intercept the 'pipeline_mode' request and orchestrate the clock domain crossing.

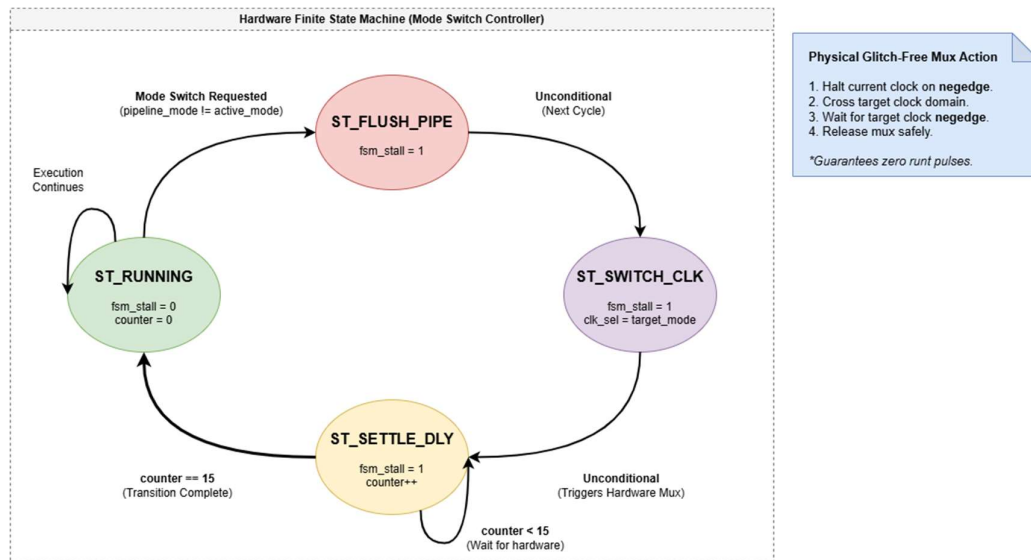


Figure 3.20: Finite State Machine (FSM) for the Glitch-Free Clock Multiplexing

When a mode switch is requested, the FSM immediately drops out of the 'ST_RUNNING' state and asserts a global 'fsm_stall' signal to freeze the processor's datapath. After passing through 'ST_FLUSH_PIPE' to clear any transient instructions, the FSM triggers the physical hardware multiplexer in 'ST_SWITCH_CLK'.

Crucially, to prevent electrical instability, the FSM then enters the 'ST_SETTLE_DLY' state. During this state, it enforces a strict 15-cycle wait period. This multi-cycle delay acts as a guard band to ensure that any asynchronous switching noise within the Lattice Phase-Locked Loop (PLL) and clock distribution network has completely dissipated before the CPU is allowed to resume operation.

The physical manifestation of this state machine on the clock distribution network is shown in the timing diagram shown in Figure 3.21 below.



Figure 3.21: Timing Diagram of the Glitch-Free Clock Transition

This waveform (Figure 3.21) illustrates the exact behavior of the glitch-free clock switching mechanism transitioning from the high-performance 5-stage mode to the energy-efficient 3-stage mode. The process relies on pure negative-edge synchronization to protect the flip-flops.

By combining the protective multi-cycle FSM with the negative-synchronization hardware, the adaptive core can dynamically shift between pipeline depths and frequencies at runtime without any risk data corruption.

3.5.3 Combinational Power Gating (Operand Isolation)

While the dual-clocking strategy effectively mitigates the frequency component of the dynamic power equation ($P_{dyn} = \alpha CV^2 f$), unnecessary switching activities can still occur in combinational logic. This toggling contributes heavily to the switching activity factor (α). In a standard pipelined architecture, if a stall is injected or a No-Operation (NOP) bubble is executed, the previous operands residing at the execution stage can still propagate through the massive combinational logic tree

of the Arithmetic Logic Unit (ALU). This unnecessary toggling burns dynamic power without producing any useful computational work or altering the architectural state.

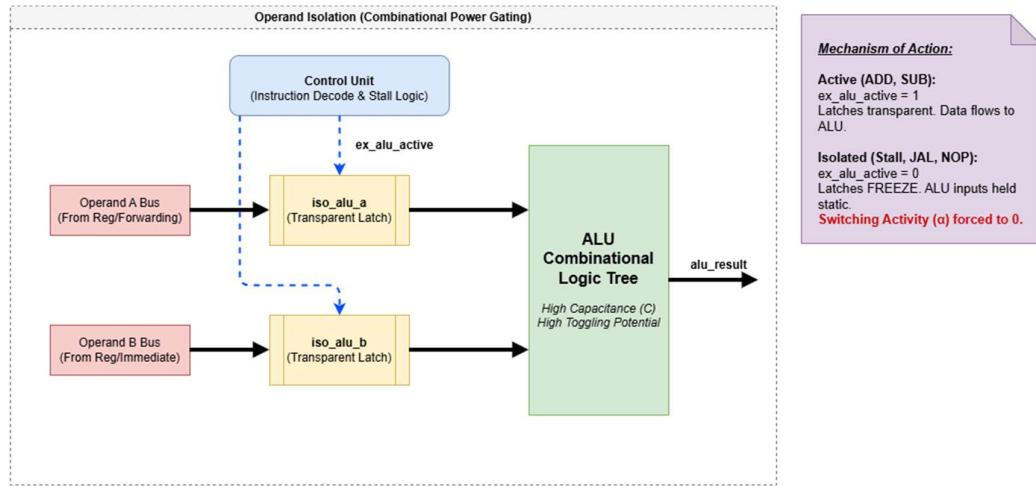


Figure 3.22: Operand Isolation Architecture

To maximize the energy efficiency of the adaptive core, operand isolation which is a form of combinational power gating was implemented within the datapath. As illustrated in Figure 3.25, dedicated hardware latches which are 'iso_alu_a' and 'iso_alu_b' were inserted as physical barriers directly before the ALU. These isolation latches are strictly controlled by an enable signal generated by the Control Unit.

During a pipeline stall or current instruction does not require arithmetic computation, the isolation latches freeze. This mechanism holds the ALU input static, cutting off the switching activity at the source. By preventing the transient signal propagation, this technique eliminates meaningless power leakage deep within the combinational logic tree, further optimizing the core for low-power applications in the 3-stage mode.

3.6 System Verification and Implementation Strategy

To ensure the architectural correctness of the adaptive RISC-V core and validate the research objectives, a multi-layered verification and implementation methodology as established. This methodology follows a bottom-up flow, moving from individual module validation to full-system hardware integration.

3.6.1 Verification Methodology: Modular to System-Level

The verification strategy was partitioned into three layers to ensure architectural correctness. This bottom-up approach allows for isolated debugging of logic blocks before validating the complex software-hardware interface and the full SoC integration.

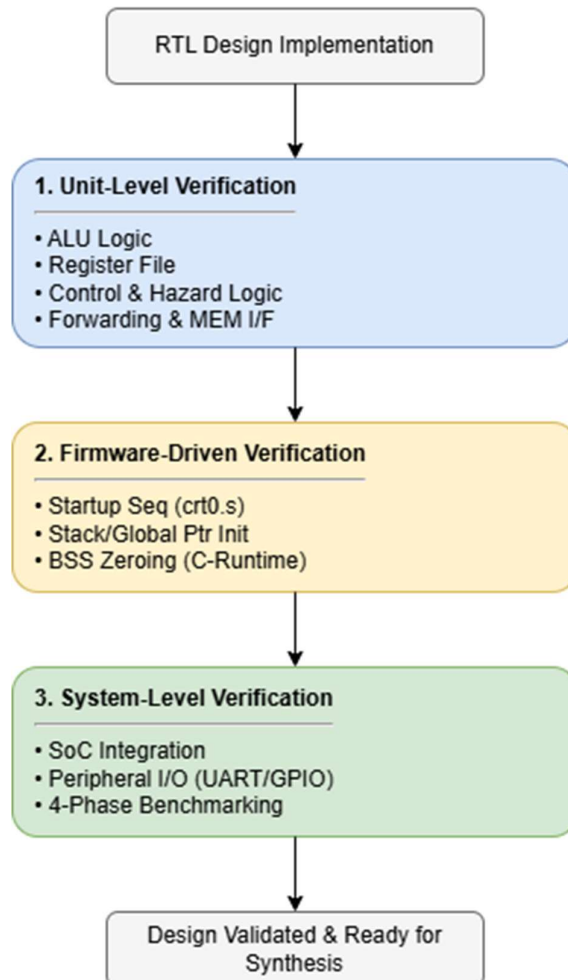


Figure 3.23: Verification Hierarchy and Implementation Flow
The verification strategy progresses from unit-level module validation, through hardware-software firmware bring-up, to final system-level SoC benchmarking.

Unit-Level Verification

This phase focused on individual component integrity using directed testbenches, ensuring that each sub-module complied with its design specification. Specifically, the Arithmetic Logic Unit (ALU) was verified against exhaustive arithmetic test vectors to ensure the correct operation of all functional units, including addition, subtraction, logical operations, and comparisons. The Register File was validated for read/write integrity by checking "Read-After-Write" (RAW) consistency and ensuring that the x0 register remained hardwired to zero. Furthermore, the Control and Hazard Logic were validated by forcing hazard conditions, such as data hazards, load-use dependencies, and branch mispredictions, to confirm the correct generation of stall and flush signals. Finally, the Forwarding Unit was tested across all pipeline bypass paths, while the memory interface was verified for the correct alignment of byte, half-word, and word operations.

Hardware-Software Integration (Firmware-Driven)

This intermediate phase served as the initial validation to ensure the core could correctly interact with the RISC-V Instruction Set Architecture (ISA) and the custom firmware environment. First, startup sequence integrity was verified by confirming the core correctly executed the crt0.s startup code. This ensured the proper initialization of the global pointer and stack pointer within the 4KB RAM bounds defined in the linker script 'link.ld'. Following this, runtime initialization was tested to confirm the core could successfully execute the BSS-clearing (Block Started by Symbol) loop. This step ensured that C-global variables were correctly zero-initialized prior to the

transition to the main application, effectively validating the core's ability to maintain architectural state across the pipeline's control logic.

System-Level Verification

Once the core was confirmed capable of executing firmware, the design was integrated into a full system-level testbench to validate the complete System-on-Chip (SoC). Within the benchmarking environment, the framework executes bare-metal C firmware utilizing a memory-mapped `tohost` communication mechanism. The testbench monitors a specific memory address (0x803F_FF00) to autonomously signal test completion, ensuring deterministic verification of the Design Under Test (DUT). This phase also validated peripheral interaction by testing the communication between the processor core and the implemented hardware peripherals such as Timer, UART, and GPIO. This confirmed that the System-on-Chip (SoC) could correctly handle memory-mapped I/O (MMIO) requests generated during firmware execution. Finally, a specialized four-phase benchmarking workload was deployed to stress specific microarchitectural domains and ensure comprehensive coverage of the core's operational logic. The suite begins with ALU arithmetic tests to verify general instruction execution and datapath integrity, followed by Data RAM access operations to validate memory controller timing. The workload subsequently tests hazard detection, stall logic, and branch resolution. These final tests confirm that the pipeline interlocks operate correctly and that the control flow safely handles architectural flushes, ultimately validating the core's overall stability.

3.6.2 FPGA Synthesis and Timing Closure

The synthesis strategy was structured to ensure hardware reliability through vendor-specific primitives utilizing the Lattice Phase-Locked Loop (PLL) IP to synthesize stable `clk\_fast` (50 MHz) and `clk\_slow` (20 MHz) domains from a 125 MHz on-board oscillator.

To achieve robust timing closure, a comprehensive Lattice Design Constraint (.ldc) file was developed to define the operational boundaries of the SoC, as illustrated in Figure 3.

```
# 1. Primary Input
create_clock -name {clk_in} -period 8 [get_ports clk_in]

# 2. Generated CPU Clock
# Using a wildcard for the output pin to avoid hierarchy errors
create_generated_clock -name {clk_slow} -source [get_ports clk_in] -divide_by 25 -multiply_by 4 [get_pins u_pll/lsec_pll_inst/clkop_o]
create_generated_clock -name {clk_fast} -source [get_ports clk_in] -divide_by 5 -multiply_by 2 [get_pins u_pll/lsec_pll_inst/clkos_o]
set_clock_groups -physically_exclusive -group [get_clocks clk_fast] -group [get_clocks clk_slow]

# 3. IO Delays
set_input_delay -clock [get_clocks clk_in] -max 2 [get_ports rst_n]
set_output_delay -clock [get_clocks clk_in] -max 2 [get_ports {leds[*]}]

# 4. False Paths
# Wildcard used to ignore timing on the PLL lock and Reset
set_false_path -from [get_pins u_pll/lock_o]
set_false_path -from [get_ports rst_n]
set_false_path -to [get_ports {leds[0] leds[1] leds[2] leds[3] leds[4] leds[5] leds[6] leds[7]}]
set_false_path -from [get_ports uart_rx]
set_false_path -to [get_ports uart_tx]

# 5. The Clock-Edge False Paths (The RAM Half-Cycle Traps)
set_false_path -rise_from [get_clocks clk_fast] -fall_to [get_clocks clk_fast]
set_false_path -fall_from [get_clocks clk_fast] -rise_to [get_clocks clk_fast]
```

Figure 3.24: Lattice Design Constraint (.ldc) File Configuration

The constraint strategy, detailed in Figure 3.27, was partitioned into three critical areas.

First, the adaptive clocking strategy defined the clock domains (`clk\_fast` and `clk\_slow`) as physically exclusive using the `set\_clock\_groups` constraint. This architectural decision informs the Static Timing Analysis (STA) engine that the core operates in either high-performance or energy-efficient mode, but never concurrently. Consequently, this eliminates unnecessary cross-domain timing analysis, allowing the synthesis tool to focus optimization strictly on the active data path.

Second, I/O delay management was implemented to ensure valid data interfacing with external components. By applying ``set_input_delay`` and ``set_output_delay`` constraints to the ``rst_n`` and ``leds`` ports, the timing engine is provided with the necessary margins for I/O signal arrival and departure.

Finally, asynchronous boundary handling was managed by isolating peripheral interfaces, such as the UART and LED controller, using ``set_false_path`` constraints. This approach prevents the STA engine from flagging timing violations on non-critical, asynchronous signals, which ultimately reduces the probability of false timing failures during synthesis.

3.6.3 Empirical Power Estimation Methodology

To validate the primary research objective of dynamic power reduction, an activity-driven power estimation methodology is employed. During the execution of the benchmark workload, the verification environment actively records the precise switching events of every internal net, register and multiplexer. These events are logged into comprehensive Value Change Dump (VCD) files for both the 5-stage and 3-stage pipeline execution passes.

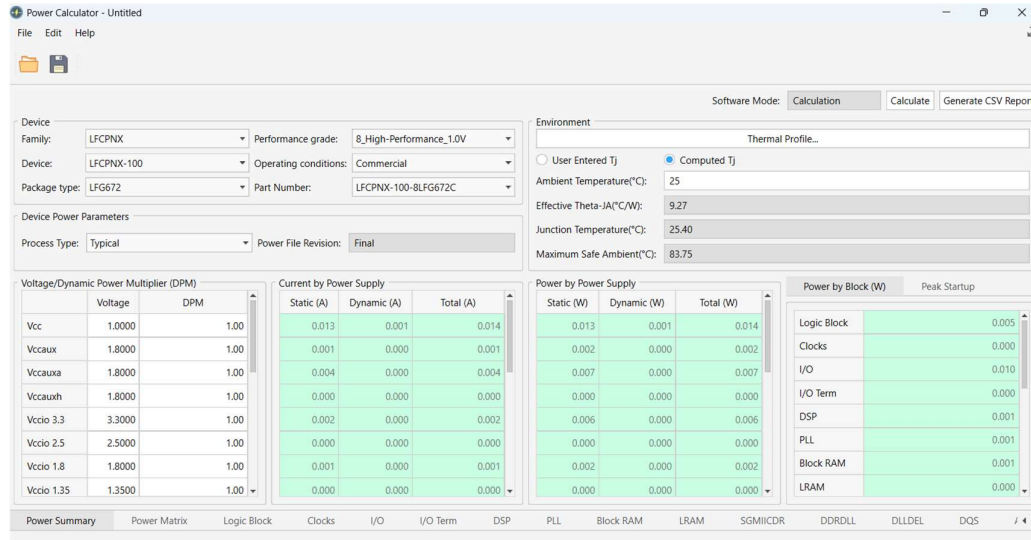


Figure 3.25: Lattice Power Calculator Environment.

These empirical activity files are then imported directly into the Lattice Power Calculator, as illustrated in Figure 3.28. The tool maps empirical toggle rates (α) extracted from VCD files onto the specific device architecture to yield hardware-validated power measurements. This flow allows for an accurate comparative analysis between the high-performance 5-stage mode and the energy-efficient 3-stage mode, finally providing the empirical basis for the results presented in the following chapter.

3.7 Chapter Summary

This chapter has detailed the systematic methodology adopted to design, verify and implement the adaptive RISC-V core. The design process centered on achieving a balance between high-performance execution and energy-efficient operation, facilitated by an adaptive 5-stage to 3-stage pipeline architecture. Key architectural innovations, including the FSM-controlled glitch-free dual-domain clocking strategy and the implementation of combinational operand isolation, were established to mitigate dynamic power consumption by addressing both the frequency and switching activity components.

To ensure the reliability of these architectural modifications, a hierarchical verification strategy was employed. This bottom-up approach began with unit-level validation of individual logic blocks, progressed to firmware-driven bring-up to verify ISA compliance and runtime initialization, and ended with extensive system-level testing to ensure architectural stability during power-state transitions.

Finally, the physical synthesis and power estimation methodology, targeting the Lattice CertusPro-NX platform, provided the framework for empirical results. By utilizing the Lattice Power Calculator through Value Change Dump (VCD) files, the established flow ensures that the power saving claims are backed by high-fidelity hardware data. This validated framework establishes the necessary foundation for empirical performance and power evaluation presented in the following chapter.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents a quantitative evaluation and microarchitectural discussion of the developed RISC-V processor across its three progressive engineering phases. The data is organized chronologically to track how hardware resources, maximum frequency, power consumption, and verification milestones change as the architecture evolves from a single-cycle baseline core to an adaptive multi-stage core. By evaluating design iteration under identical constraints, the absolute hardware overhead and efficiency gains introduced by runtime pipeline scaling are isolated and quantified.

All architectural phases were developed in register-transfer-level (RTL) Verilog, functionally verified via cycle-accurate simulation frameworks, and synthesized for the 28nm Fully-Depleted Silicon-on-Insulator (FD-SOI) Lattice CertusPro-NX FPGA platform. Compilation, placement, routing, and static timing closure analyses were performed natively using the Lattice Radiant software suite. This chapter establishes the performance baselines of the initial design configurations, building the empirical foundation for the comparative efficiency benchmarks detailed in Chapter 5.

4.2 Baseline Performance of Single-Cycle RV32I Processor

The initial phase of development focused on the construction of a baseline single-cycle RV32I processor core. This single-cycle core serves as the golden reference for functional correctness, establishing the baseline for hardware resource

utilization and maximum combinational path delay before the introduction of pipelining.

4.2.1 Synthesis and Hardware Resource Utilization Profile

The baseline single-cycle core was synthesized, placed, and routed using an area-optimization strategy within the Lattice Radiant software environment. Table 4.1 presents the gate-level structural footprint data extracted from the post-route synthesis reports.

Table 4.1: Single-Cycle Core Hardware Resource Utilization

<i>Resource Types</i>	<i>Utilized Macro Cells</i>	<i>Total Available</i>	<i>Allocation Percentage (%)</i>
Look-Up Tables (LUT4)	2,348	84,255	2.78%
Dedicated Flip-Flops (DFF)	161	84,255	0.19%
Programmable Half-Adders	52	42,127	0.12%
Embedded Block RAM (EBR)	64	346	18.49%
Hardware Multipliers (DSP)	0	56	0%

The synthesis data reveals an extreme structural asymmetry between the combinational logic footprint (2,348 LUT4 macro blocks) and the sequential storage footprint (161 DFF elements). This structural variance serves as a definitive microarchitectural indicator of an unsegmented single-cycle datapath. Because the entire instruction execution cycle spanning the instruction fetch (IF), instruction decode (ID), arithmetic execution (EX), data memory access (MEM), and register write-back (WB) phases must resolve combinational delays structurally within one

clock edge, intermediate pipeline stage boundary registers are entirely absent from the netlist design.

The sequential allocation is restricted to a minimal footprint of 161 DFF elements. It is critical to note that this register allocation cannot accommodate a standard flip-flop-based implementation of the general-purpose register file, which would require 992 discrete flip-flops to house registers x_1 through x_{31} (31 registers X 32 bits). Instead, the Lattice Radiant synthesis engine automatically inferred the asynchronous-read register file array as distributed LUT-RAM primitives, utilizing the combinational look-up table structures as shallow memory storage cells. Consequently, the 161 DFF primitives are strictly reserved for the 32-bit Program Counter (PC) block and the internal control and state-tracking registers embedded within the integrated SoC peripherals. Conversely, register x_0 is hardwire combinationally directly to ground, removing it completely from the sequential and distributed RAM allocation.

The implementation metrics also indicate that 64 Embedded Block RAM (EBR) arrays are consumed by architecture. Based on raw address-space configurations, the concurrent allocation of a 32K X 32-bit instruction ROM (128 KB) and a 32K X 32-bit data RAM (128 KB) represent a combined memory footprint of 256KB, which theoretically demands 114 EBR blocks based on the 18 Kb capacity of the physical SP18KD hard macros. The post-route synthesis of exactly 64 EBR blocks indicates that the compiler executed automated pruning and address-space optimization during the design elaboration phase. Because the compiled bare-metal verification software payload does not span the maximum addressable depth of the allocated 32K word arrays, the synthesis engine automatically removes unpopulated high-address storage blocks to minimize physical macro overhead. Utilizing these

dedicated hardware blocks prevents the compiler from spilling active memory structures into the distributed fabric logic (LUT-RAM), thereby preserving look-up tables for core operational execution logic and routing networks. Furthermore, the 0% utilization of dedicated hardware multipliers DSP indicates that the baseline arithmetic logic unit (ALU) was synthesized entirely out of general fabric LUT4 cells. This confirms that all logic operations, barrel shifters, and carry-chains are routed through regular logic cells, defining a lean structural baseline for the initial configuration.

4.2.2 Timing and Propagation Path Analysis

Static Timing Analysis (STA) was executed on the post-route netlist under worst-case operating parameters to isolate the structural timing boundaries of the unsegmented single-cycle datapath. Table 4.2 documents the compiled timing limits, operational targets, and computational throughput metrics of the baseline core.

Table 4.2: Single-Cycle Core Timing and Performance Profile

<i>Performance Metric</i>	<i>Quantified Value</i>
Maximum Theoretical Frequency (F_{max})	29.109 MHz
Worst-Case Critical Path Propagation Delay (T_{crit})	34.353 ns
Benchmarked Operational Frequency (F_{bus})	20.000 MHz
Structural Cycles Per Instruction (CPI)	1.000
Active Computational Throughput	20.000 MIPS

The post-route Static Timing Analysis (STA) confirms that the single-cycle processor architecture is severely frequency-bounded, achieving a maximum theoretical frequency (F_{max}) of 29.109 MHz. This boundary is enforced by the worst-

case critical path propagation delay (T_{crit}) calculated at 34.353 ns. To ensure functional stability across the fabric and prevent setup or hold violations during real-world execution, the physical evaluation clock target was backed off to a synchronous target of 20.000 MHz. At this operational rate, the core maintains a steady computational throughput of 20.000 MIPS due to its structural execution model, which maintains a constant CPI factor of exactly 1.000 across all instruction classes.

According to the post-route timing report, the absolute critical path bottleneck is explicitly localized within the load-word (LW) instruction data loop. Rather than a general distribution of gate delays, the timing analyzer identified that the worst-case setup slack boundary stretches continuously from the edge-triggered Program Counter primitives, through the address ports of the instruction memory, across the asynchronous read path of the register file, through the carry-chain propagation cells of the ALU adder, and finally terminates at the input synchronous write features of the data memory block.

Because entire instruction processing loop must resolve within one uniform clock edge to maintain the 1.000 CPI, the accumulated combinational path delay establishes a severe frequency ceiling. This post-route timing constraint provides the direct empirical justification for the multi-stage pipeline modification developed in the subsequent design phases, which seek to segment this continuous logic path into discrete clock-bounded intervals.

4.2.3 Power Characterization and Dissipation Analysis

Post-route power metrics for the baseline single-cycle core were generated by logging switching activity to a Value Change Dump (.vcd) file during cycle-accurate simulation, which was then analyzed using the Lattice Power Calculator suite. These

active switching activities were annotated under a high-workload scenario featuring a deterministic 62% input node activity factor at the benchmarked operating frequency of 20.000 MHz. Table 4.3 outlines the resulting power distribution breakdown.

Table 4.3: Baseline Single-Cycle Core Power Dissipation Parameters

<i>Power Parameter Component</i>	<i>Quantified Value (mW)</i>	<i>Budget Percentage (%)</i>
Static Baseline Leakage Power (P_{stat})	43.000 mW	71.67%
Active Dynamic Switching Power (P_{dyn})	16.000 mW	26.67%
Routing and Input/Output Pad (P_{io})	1.000 mW	1.66%
Total Core Power Dissipation (P_{total})	60.000 mW	100.00%

The post-route power profile indicates that static baseline leakage power (43.000 mW) dominates the core energy footprint, accounting for 71.67% of the total 60.000 mW power budget. Active dynamic switching power (16.000 mW) constitutes 26.67% of the total envelope, while structural routing interconnect nets and physical I/O pads account for the remaining 1.66% (1.000 mW). To visually contextualise this budgetary allocation, the proportional metrics are plotted in Figure 4.1.

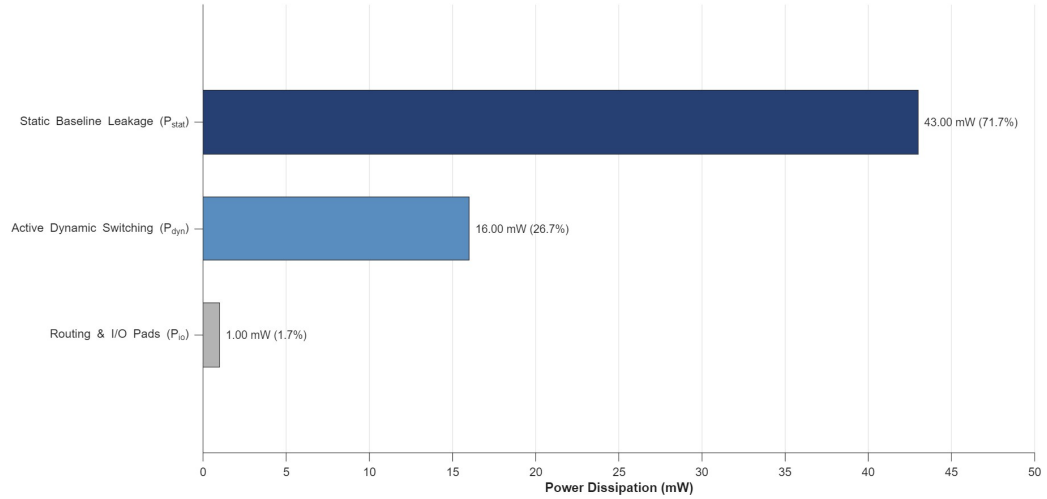


Figure 4.1: Proportional Power Dissipation Components of the Baseline Single-Cycle Core.

As illustrated in Figure 4.1, the clear dominance of static power over active switching power is a direct sequence of the physical scale mismatch on the FPGA fabric prototyping environment. Static leakage power within the 28 nm FD-SOI substrate is governed by subthreshold leakage and gate oxide leakages across all active and idle transistor networks inside the hardware layout boundaries. Because this compact 2,348 LUT single-cycle processing core is mapped onto the expansive fabric layout of the Lattice CertusPro-NX LFCPNX-100 device which contains 84,255 available Look-Up Tables, the background leakage of the unallocated, un-gated silicon cells naturally outweighs the localized switching currents of the active design nets.

Furthermore, due to the unsegmented architecture timing-bounded to a low operational frequency, 20.000 MHz, the total capacitive toggle rate across the active combinational network path is heavily restricted, preventing the dynamic power consumed from scaling up.

4.3 Baseline Performance of 5-Stage Pipeline RV32I Processor (Phase 2)

Following the characterization of the baseline single-cycle core reference, the unsegmented datapath migrated to a synchronous 5-stage pipelined configuration. This architectural iteration separated the continuous combinational logic path into five distinct, clock-bounded execution zones: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). These zones are isolated by edge-triggered stage boundary registers. This section evaluates the resulting shifts in fabric resource utilization, post-route timing closure, and the execution penalties introduced by pipeline hazards.

4.3.1 Hardware Resource Utilization and Fabric Overhead

The static 5-stage pipeline core was compiled under an area-optimisation constraint inside the Lattice Radiant toolflow. Table 4.4 presents the post-route macro primitive metrics extracted from the design summary layout.

Table 4.4: Static 5-Stage Pipeline Resource Utilization

<i>Resource Types</i>	<i>Utilized Macro Cells</i>	<i>Total Available</i>	<i>Allocation Percentage (%)</i>
Look-Up Tables (LUT4)	2,899	84,255	3.44%
Dedicated Flip-Flops (DFF)	933	84,255	1.11%
Programmable Half-Adders	128	42,127	0.30%
Embedded Block RAM (EBR)	66	346	19.08%
Hardware Multipliers (DSP)	0	56	0%

The synthesis profile indicates a substantial structural shift in the hardware footprint compared to the single-cycle baseline. While the combinational logic

footprint increased by 23.46% to 2,899 LUT4 blocks, the sequential register footprint experienced a sharp increase of 479.50%, expanding from 161 to 933 DFF elements. This register explosion represents the direct physical cost of pipelining the datapath.

The additional 772 DFF primitives are completely consumed by the synchronous stage isolation registers alongside the internal pipeline state registers required by the newly integrated data forwarding logic and hazard stall units.

The synthesis report also documents an increment of two additional EBR macro blocks to 66 blocks in total. This minor expansion is attributed to the automated replication of block select lines and dual-port byte-enable logic paths generated by the compiler to manage simultaneous instruction fetching and data bus interactions across separate pipeline stages without introducing a structural memory hazard. The DSP allocation still remains at 0%, confirming that the pipeline’s operational logic continues to be constructed entirely out of regular programmable cell blocks.

4.3.2 Static Timing Analysis and Frequency Closure

Static Timing Analysis (STA) was conducted on the post-route netlist to evaluate the timing performance of the segmented datapath. By isolating each stage boundary behind edge-triggered registers, the worst-case combinational path delay is no longer bounded by the cumulative delay of the entire processor, but rather by the latency of the slowest single stage. Table 4.5 details the resulting timing and performance metrics.

Table 4.5: Static 5-Stage Pipeline Timing and Performance Profile

<i>Performance Metric</i>	<i>Quantified Value</i>
Maximum Theoretical Frequency ($F_{\max}$)	54.347 MHz

Worst-Case Critical Path Propagation Delay (T_{crit})	18.400 ns
Benchmarked Operational Frequency (F_{bus})	50.000 MHz
Nominal Peak Computational Throughput	50.000 MIPS

The post-route STA reports confirm successful timing closure, with the maximum theoretical frequency (F_{max}) skyrocketing to 54.347MHz, an 86.70% frequency enhancement over the single-cycle floor. This frequency leap is driven by the reduction of the worst-case critical path propagation delay (T_{crit}) down to 18.400 ns. Consequently, the benchmarked operational frequency target was safely locked at a synchronous baseline of 50.000 MHz.

The localized critical path bottleneck is no longer found in the continuous load-word data loop but is instead confined entirely within the Execute (EX) stage. Specifically, the critical path delay is bounded by the propagation delay through the forwarding multiplexers, followed by the deep combinational carry-chain logic within the 32-bit Arithmetic Logic Unit adder block during sign-extension operations, before terminating at the setup boundary of the pipeline register interface between Execute (EX) stage and Memory Access (MEM) stage.

4.3.3 Power Characterization and Dissipation Profile

To complete the standard Power, Performance, and Area (PPA) profiling of the 5-stage architecture, power estimations were evaluated using the Lattice Power Calculator. Driven by the 50.000 MHz benchmark execution clock, the static 5-stage pipeline yielded a total core power dissipation (P_{total}) of 84.000 mW. Table 4.6 outlines the top-level power distribution profile across the device fabric.

Table 4.6: Static 5-Stage Pipeline Power Dissipation Summary

<i>Power Parameter Component</i>	<i>Quantified Value (mW)</i>	<i>Budget Percentage (%)</i>
Static Baseline Leakage Power (P_{stat})	42.000 mW	50.00%
Active Dynamic Switching Power (P_{dyn})	41.000 mW	48.81%
Routing and Input/Output Pad (P_{io})	1.000 mW	1.19%
Total Core Power Dissipation (P_{total})	84.000 mW	100.00%

The transition from a single-cycle core to a 5-stage pipelined architecture triggers a substantial shift in the power dissipation budget. In the single-cycle baseline, active dynamic power accounted for only 26.67% of the total budget. However, as detailed in Table 4.6, the 5-stage pipeline’s dynamic switching power (P_{dyn}) surges to 41.000 mW, consuming nearly half (48.81%) of the entire processor power budget.

This phenomenon is a direct microarchitectural consequence of pipelining. The massive integration of 933 sequential DFF primitives forming the stage boundary registers, data forwarding networks and structural interlock units creates high capacitive loads. Because these registers are constantly toggle at 50.000 MHz to latch instruction and data states, dynamic power consumption increases significantly.

4.3.4 Architectural Compliance and Pipeline Hazard Penalties

To validate the architectural correctness of the newly introduced hazard-mitigation logic, formal functional verification was orchestrated via the RISCOF framework within the WSL2 environment. The 5-stage core successfully achieved 100% compliance alignment, passing all 39 standard architectural instruction test

suites within the base RV32I specification by matching its memory signature dumps byte-for-byte against the Spike reference model.

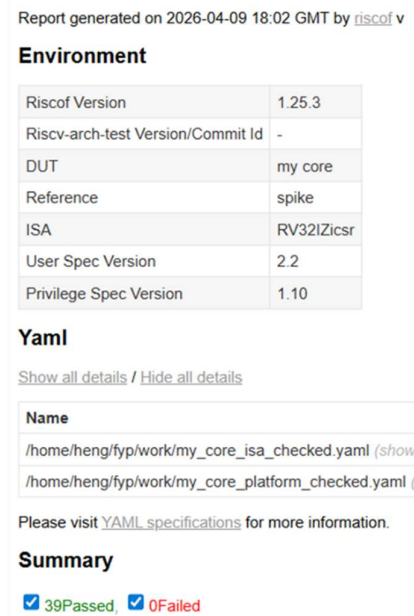


Figure 4.2: 100% Compliance of the 5-Stage Pipeline Architecture with the RISCOF RV32I Test Suite against the Spike Reference Model

However, while the high clock speed delivers a peak theoretical execution throughput of 50.000 MIPS at 50 MHz, the structural dependencies of the 5-stage pipeline introduce hazard penalties that degrade the real-world Cycles Per Instruction (CPI) factor and computational instruction throughput across different instruction workloads. Table 4.6 quantifies the microarchitectural hazard penalties and active execution throughput evaluated during cycle-accurate benchmark execution loops within QuestaSim.

Table 4.7: 5-Stage Pipeline Hazard Penalty and Throughput Breakdown

<i>Instruction Class</i>	<i>Hazard and Penalty Condition</i>	<i>Effective CPI</i>
ALU-Register	Minor data dependency tracking and write-back register synchronization latency.	1.143
Memory Store	Simultaneous EX base address generation and source register forwarding interlock.	1.200

Memory Load	1 clock cycle structural load-use hazard interlock bubble injected into the pipeline.	1.600
Control Branch	2 clock cycle instruction pipeline flush upon late branch execution mismatch.	1.500

The empirical data compiled in Table 4.6 isolates the exact throughput penalties introduced by real-world pipeline hazards, demonstrating why the core's operational efficiency deviates from an ideal unsegmented execution profile.

ALU-Register Dependency Overhead (CPI = 1.143)

Despite the integration of a hardware data forwarding network, consecutive register-to-register instruction streams experience fractional latencies. This overhead triggered from cumulative propagation delays across the multi-stage forwarding multiplexer selection networks. This structural penalty restricts the realized computational throughput to 43.750 MIPS.

Memory Store Interlock Penalty (CPI = 1.200)

For data store instructions ('SW'), calculating the target memory address base-offset within the Execute (EX) stage while simultaneously fetching the data payload from a source register creates an operand routing bottleneck. Because the ALU adder cannot resolve the address arithmetic while the steering networks handle an active forwarding path for the data payload that needs to be stored concurrently within the same clock cycle, the hazard unit introduces a temporary interlock cycle to prevent path contention, limiting the output to 41.670 MIPS.

Load-Use Hazard Penalty (CPI = 1.600)

For memory load instructions ('LW'), data fetched from the external synchronous Embedded Block RAM (EBR) macro does not settle on the data bus until the completion of the Memory Access (MEM) stage. When an immediately succeeding instruction depends on this target register, a structural hazard occurs that data forwarding wires cannot resolve due to the data fetched not yet exist on the location. The hazard detection unit is forced to assert a master pipeline stall to freeze the Program Counter (PC) and injecting a 1-cycle NOP bubble into the datapath which degrades the execution throughput to its absolute baseline floor of 31.250 MIPS.

Control Hazard Penalty (CPI = 1.500)

A branch prediction mismatch triggers a structural hazard due to the conditional branches evaluating their target conditions late within the EX stage via the dedicated branch comparator module. The control logic asserts a master pipeline flush and completely clears the speculative instructions fetched ahead in the Instruction Fetch (IF) and Instruction Decode (ID) stages. The destruction of these two execution states transforms the cycles into NOP bubbles, restricting active throughput to 33.330 MIPS.

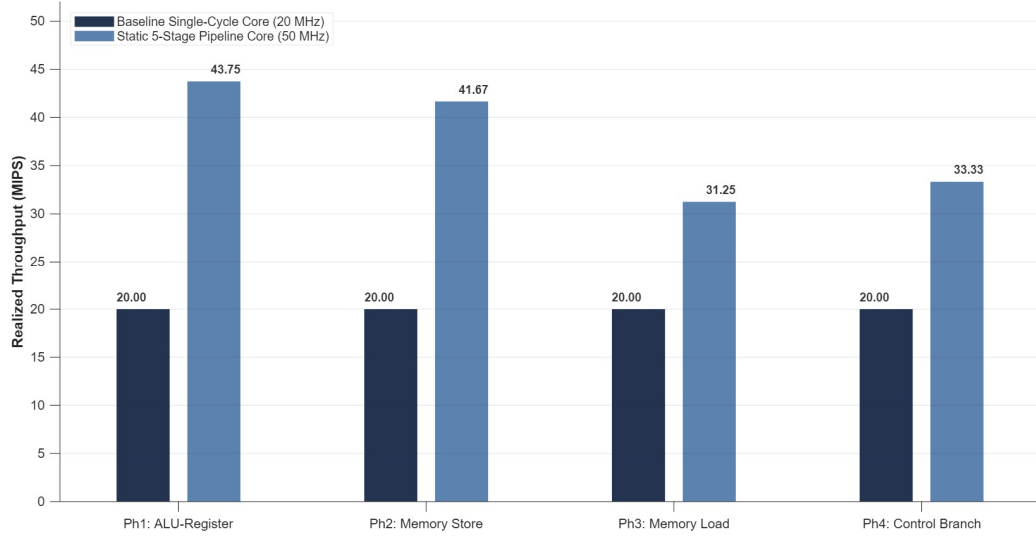


Figure 4.3: Realized MIPS Throughput Comparison across Different Instruction Test Workloads.

While the 5-stage pipelined core faced massive increase in clock frequency (50.000 MHz) ensures that the net MIPS throughput remains significantly higher than the single-cycle baseline across all phases, these penalties degrade hardware runtime efficiency.

4.4 Performance Evaluation of Software-Reconfigurable 3/5-Stage Adaptive Processor (Phase 3)

Following the characterization of the fixed 5-stage pipelined processor, it was migrated to a software-reconfigurable, adaptive 3/5-stage architecture. This design introduces runtime structural elasticity, allowing the core to dynamically alter its pipeline depth via software execution commands. This section evaluates the resulting synthesis footprint overhead and workload-driven dynamic power scaling.

4.4.1 Control Logic Footprint and Structural Area Overhead

The adaptive core was compiled under an area-optimization constraint inside the Lattice Radiant toolflow. Table 4.7 presents the physical post-route macro cell metrics.

Table 4.8: Adaptive Multi-Stage Core Hardware Resource Utilization

<i>Resource Types</i>	<i>Utilized Macro Cells</i>	<i>Total Available</i>	<i>Allocation Percentage (%)</i>
Look-Up Tables (LUT4)	4,409	84,255	5.23%
Dedicated Flip-Flops (DFF)	2,011	84,255	2.39%
Programmable Half-Adders	184	42,127	0.44%
Embedded Block RAM (EBR)	66	346	19.08%
Hardware Multipliers (DSP)	0	56	0%

Integrating runtime reconfigurability introduces a noticeable hardware footprint penalty across the FPGA fabric. Compared to the static 5-stage pipeline baseline, the combinational logic footprint expanded by 52.09% to 4,409 LUT4 cells, while the sequential storage allocation surged by 115.54% to 2,011 DFF elements. This structural area overhead is explicitly driven by three microarchitectural additions:

1. ***The Mode Transition Finite State Machine (FSM)***: A centralized hardware controller that monitors execution boundaries and orchestrates safe pipeline drainage. During mode transitions, this four-state FSM manages the master stall sequences and handshake signals required to switch clock domains and isolate inactive state registers without corrupting in-flight instruction data.

2. ***Elastic Multiplexing and Bypassing Networks***: An extensive array of wide data-steering logic cells embedded at the structural stage boundaries. When the processor scales down into the highly efficient 3-stage mode, these multiplexers physically bypass the intermediate Execute (EX) and Memory Access (MEM) boundary registers, instantaneously rerouting 32-bit datapaths, ALU results, and control signals directly to the final Write-Back (WB) logic.
3. ***Glitch-Free Clock Multiplexing and Operand Isolation Circuits***: To securely shift between the 50 MHz and 20 MHz clock trees without introducing metastable condition, dedicated dual-edge synchronizer flip-flops and clock-enable primitives are synthesized into the fabric. Additionally, combinational clamping gates and intermediate barrier registers are instantiated to force the ALU inputs to zero, physically blocking signal toggles from propagation through the idle pipeline zones.

4.4.2 Workload Execution Throughput Across Different Modes

To verify architectural compliance and track execution throughput during reconfiguration, cycle-accurate simulation was executed across both the high-performance 5-stage mode (operating at 50.000 MHz) and the energy-saving 3-stage mode (clocked down to 20.000 MHz). Table 4.9 documents the benchmarked performance data extracted from the QuestaSim post-route validation signatures.

Table 4.9: Adaptive Core Throughput Profiling Across Reconfiguration Modes

<i>Instruction Class</i>	<i>5-Stage Mode</i>	<i>3-Stage Mode</i>
ALU-Register	1.143	1.143
Memory Store	1.200	1.200
Memory Load	1.600	1.600

Control Branch	1.500	1.500
-----------------------	-------	-------

The functional data in Table 4.9 proves that the adaptive core maintains complete mathematical correctness across both configurations, hitting a 100% pass status. Due to the same internal forwarding networks and interlock logic remaining physically present in the netlist, the effective CPI factors stay identical between the two modes.

To quantify how these identical CPI translate into real-world computing speeds, the microarchitectural execution limits must be mapped directly against the operational clock domains. Table 4.10 outlines the timing boundaries and peak computational scaling across both reconfigurable software states.

Table 4.10: Adaptive Core Timing and Performance Profile

<i>Performance Metric</i>	<i>5-Stage</i>	<i>3-Stage</i>
Maximum Theoretical Frequency ($F_{\max}$)	54.347 MHz	24.914 MHz
Benchmarked Operational Frequency (F_{bus})	50.000 MHz	20.000 MHz
Nominal Peak Computational Throughput	50.000 MIPS	20.000 MIPS
Realized Workload Throughput (ALU-Register)	43.750 MIPS	17.500 MIPS

However, because the 3-stage mode is clocked at a lower frequency of 20.000 MHz to achieve a stable, low-energy state, its raw computational throughput drops by exactly 60% compared to the 5-stage mode. For instance, the ALU workload drops from 43.750 MIPS down to 17.500 MIPS, and the Load workload drops from 31.250 MIPS to 12.500 MIPS.

4.4.3 Workload-Aware Dynamic Power Reduction

Post-route power estimations were conducted by feeding the mode-specific workload .vcd files into the Lattice Power Calculator. Table 4.11 outlines the dynamic switching power distribution to contrast the savings against the raw performance loss.

Table 4.11: Dynamic Power Dissipation Comparison Across Processing Modes

<i>Functional Block</i>	<i>5-Stage Mode (mW)</i>	<i>3-Stage Mode (mW)</i>	<i>Power Reduction (%)</i>
Gated State Registers	24.500 mW	0.000 mW	100.00%
Arithmetic Logic Unit	18.200 mW	6.100 mW	66.48%
Active Pipeline Control & Routing	16.300 mW	14.900 mW	8.59%
Total Active Dynamic Component (P_{dyn})	59.000 mW	21.000 mW	64.41%

The power breakdown compiled in Table 4.11 shows that the adaptive core successfully satisfies its low-power objectives, yielding a 64.41% drop in active dynamic power when switching into 3-stage mode. This substantial energy savings is achieved through two complementary hardware gating methods managed by the transition finite state machine (FSM). The first method, which is coarse-grained clock gating, is triggered when the core drops into the 3-stage configuration. During this transition, the FSM pulls the clock enable signals low for the `reg\_id\_ex` and `reg\_ex\_mem` pipeline registers. This action immediately eliminates all capacitive toggling within those deep internal register blocks. This action immediately eliminates all capacitive toggling within those deep internal register blocks, effectively reducing their dynamic power footprint to nearly 0.000 mW.

Furthermore, this efficiency is compounded by the second method which is combinational gating. In the 3-stage mode, input clamping blocks are utilized to force the Arithmetic Logic Unit (ALU) inputs to zero whenever the unified execution zone is clear of active operations. This isolation mechanism successfully block unnecessary transient signal toggles from propagating through the adder carry chains, which consequently drops the ALU's dynamic power dissipation by an impressive 66.48%.

4.5 Comprehensive Architectural Comparison and Efficiency Metrics

To evaluate the engineering value of the software-reconfigurable architecture, the raw performance and power metrics of the three design phases are Baseline Single-Cycle, Static 5-Stage, Adaptive 5-Stage Mode, and Adaptive 3-Stage Mode must be contextualized. Because clock frequency scales linearly with dynamic power ($P_{dyn} \propto f$), simply comparing raw milliwatt dissipation is insufficient.

4.5.1 Cross-Architectural Efficiency Evaluation

To properly benchmark the architectures, raw power must be weighed against execution speed using Dynamic Energy Instruction (E_{dyn}) and the Energy-Delay Product (EDP).

To quantify computational efficiency regardless of the operating clock frequency, Dynamic Energy Instruction, E_{dyn} is calculated. This metric defines how much active switching energy is consumed to retire a single instruction, expressed in nano Joules per instruction:

$$E_{dyn} = \frac{P_{dyn}}{Throughput (MIPS)} \quad (4.1)$$

While E_{dyn} measures raw energy efficiency, it does not account for the execution time penalty of running at a slower frequency. To measure the trade-off

between power savings and performance loss, the Energy-Delay Product (EDP) is evaluated. A lower EDP signifies a more optimal balance of speed and energy consumption. Rather than presenting raw ESP values in microscopic Joules-seconds, the architectures are compared using a relative, dimensionless EDP factor derived proportionally from the active power and squared throughput:

$$\text{Relative EDP Factor} \propto \frac{P_{\text{dyn}}}{\text{Throughput}^2} \quad (4.2)$$

Applying the equations (4.1) and (4.2), Table 4.12 aggregates the active dynamic power, nominal ALU execution throughput, and the resulting efficiency figures of merit across the four architectural states.

Table 4.12: Cross-Architectural Figure of Merit Comparison

<i>Architecture</i>	<i>Single-Cycle</i>	<i>Static 5-Stage</i>	<i>Adaptive 5-Stage</i>	<i>Adaptive 3-Stage</i>
Operational Frequency	20.000 MHz	50.000 MHz	50.000 MHz	20.000 MHz
Dynamic Power (P_{dyn})	16.000 mW	41.000 mW	59.000 mW	21.000 mW
Throughput	20.000 MIPS	36.740 MIPS	36.740 MIPS	14.700 MIPS
Energy Per Instruction (E_{dyn})	0.800 nJ/instr	1.116 nJ/instr	1.606 nJ/instr	1.429 nJ/instr
Relative EDP Factor	0.040	0.030	0.044	0.097

The mathematical data in Table 4.12 reveals several critical microarchitectural realities when subjected to instruction workloads:

1. **The Performance-Power Trade-Off:** The baseline single-cycle core gains the absolute lowest E_{dyn} (0.800 nJ/instr) because it lacks intermediate pipeline registers, meaning zero clock-tree power is spent latching data between stages. Furthermore, because it does not suffer from pipeline hazards, it sustains a perfect 20.000 MIPS. However, its unsegmented critical path permanently

traps it at a lower frequency ceiling, making it physically unfit for high-performance deployment.

2. **The Efficiency of Static Pipelining:** The Static 5-Stage pipeline proves to be the most optimal architecture for sustained computing. While its dynamic power increases to 41.000 mW and pipeline hazards degrade its throughput to 36.740 MIPS, its high processing power still yields the lowest EDP factor which is 0.030. This proves that deep pipelining provides the best balance of speed and power when operating at full utilization.
3. **The Tax of Adaptability:** Comparing the Static 5-Stage core to the Adaptive core running in 5-stage mode highlights the physical cost of flexibility. While both achieve identical execution throughput, the Adaptive 5-Stage mode consumes significantly higher dynamic power. This energy penalty drives its E_{dyn} up to 1.606 nJ/instr and degrades its EDP to 0.044, making it slightly worse in raw EDP than even the single-cycle baseline. This penalty is directly attributed to the elastic multiplexing networks for mode transitions.
4. **The Efficiency Recovery:** The true advantage of the Adaptive core is realized when it drops into the 3-stage mode. By intentionally sacrificing throughput during low-demand periods, its E_{dyn} improves from 1.606 nJ/instr down to 1.429 nJ/instr compared to its high-power 5-stage counterpart. By gating the deep pipeline registers and isolating the ALU inputs, the architecture successfully claws back 11% of its energy efficiency.

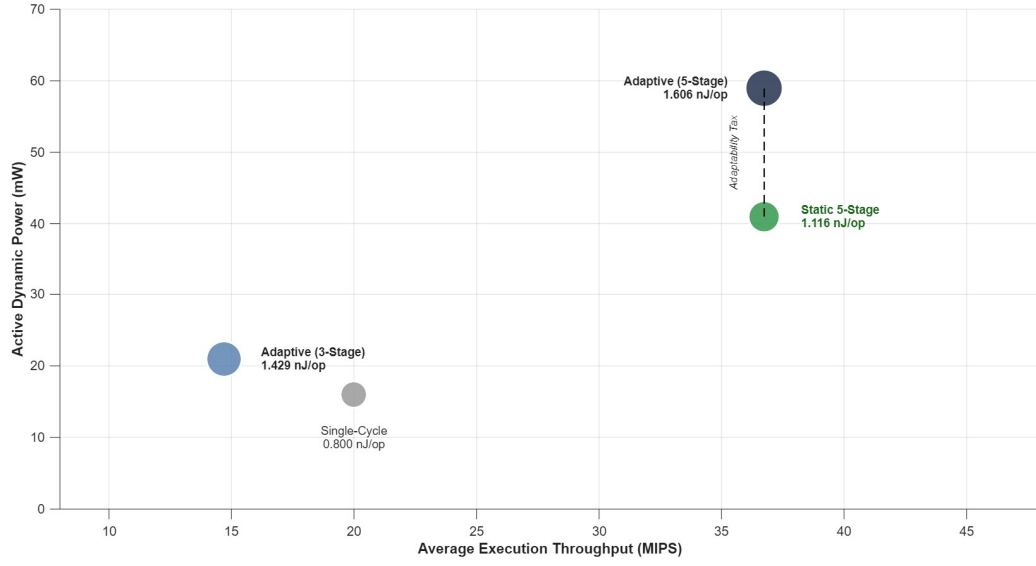


Figure 4.4: Architectural Efficiency Comparison across four hardware configurations

4.5.2 Workload Suitability and Scenario Analysis

To justify the silicon area overhead and the adaptability tax paid from adaptive 5-stage observed in Table 4.12, the architecture must be evaluated across a parametric spectrum of real-world computing scenarios rather than isolated, maximum-utilization benchmarks.

In modern edge-computing and Internet of Things (IoT) applications, processors rarely sustain 100% utilization. Instead, they operate on a skewed duty cycle, rapidly alternating between an active processing phase and a low-power idle phase.

By defining the active execution window as the duty cycle ratio (D), and the idle window as the remainder ($1 - D$), the average dynamic power dissipation (P_{avg}) for both architectures can be modeled as a relational function.

If executed on the rigid 5-Stage processor, the core lacks structural elasticity. It is forced to run continuously at 50.000 MHz, drawing its full 41.000 mW regardless of the actual computational demand. Its average dynamic power dissipation remains mathematically flat and constant across all duty cycles:

$$P_{avg(STATIC)} = 41.000 \text{ mW} \quad (4.3)$$

Conversely, the Software-Reconfigurable Adaptive Core actively scales its structure to match the workload. During the active phase, it runs in 5-Stage mode, consuming 59.000 mW to meet the high execution throughput demand. During the idle phase, the firmware forces the core into 3-Stage mode, dropping the clock to 20.000 MHz and halting the bypassed stage registers to achieve a deep-sleep power footprint of just 21.000 mW. The temporal relationship of these operational states is visualized in Figure 4.5 below.

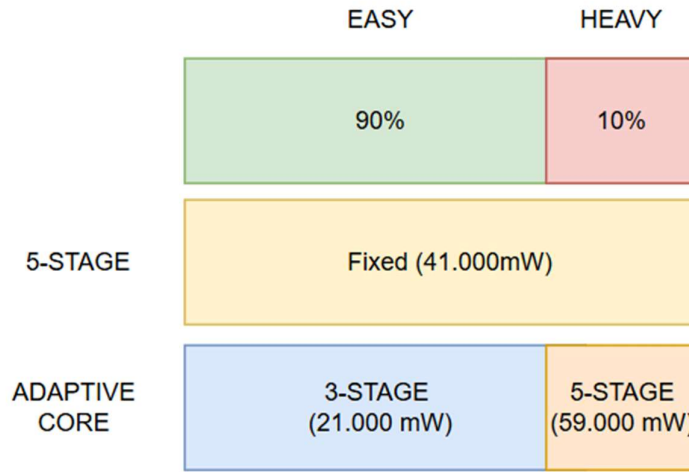


Figure 4.5: Power Consumption of both 5-Stage and Adaptive cores across variable workload

Therefore, the average power dissipation for the adaptive core forms a linear progression based on workload intensity:

$$P_{avg(ADAPTIVE)} = (59.000 \text{ mW} \times D) + (21.000 \text{ mW} \times (1 - D)) \quad (4.4)$$

By equating these two parametric models, an architectural break-even point is established at exactly $D = 52.63\%$, as illustrated in Figure 4.6.

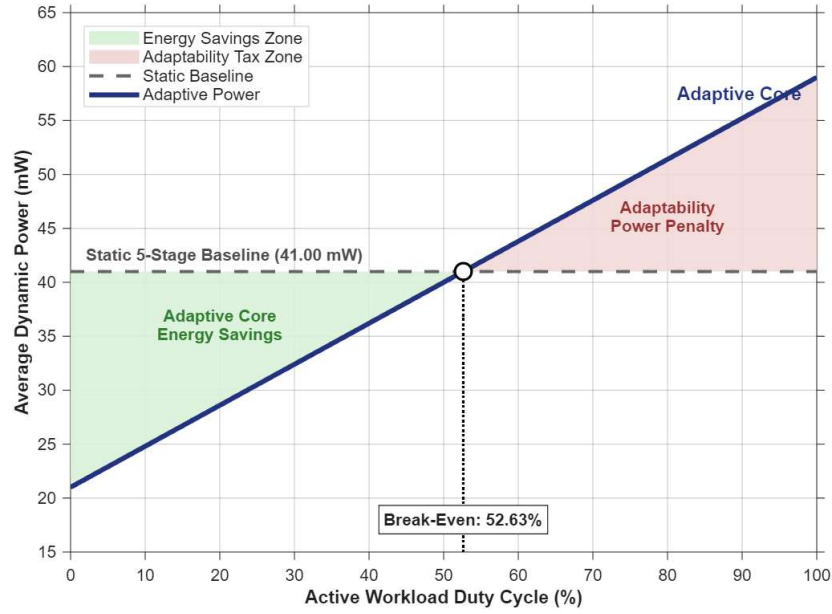


Figure 4.6: Relational Average Power Dissipation Models across Variable Active Duty Cycle

This mathematical boundary visually and quantitatively defines the specific workload suitability limits for both physical configurations. For continuous data-streaming workloads that require active processing more than 52.63% of the time ($D > 52.63\%$), the static 5-stage pipeline proves superior. Above this high-performance threshold, the initial 59.000 mW adaptability tax of the reconfigurable core outweighs its idle energy savings. This operational shift ultimately drives the adaptive design into a power penalty phase, visually represented by the red sections in Figure 4.7.

Conversely, for edge-computing scenarios operating below this threshold ($D < 52.63\%$), the adaptive core vastly outperforms the static baseline. In applications such as background sensor monitoring, the active duty cycle typically falls below 20%. Under a highly realistic 10% active duty cycle, the adaptive core achieves an average power footprint of just 24.800 mW. This yields a massive 39.51% overall energy savings compared to the static baseline's rigid 41.000 mW drain.

Ultimately, this relational analysis proves that while runtime structural elasticity introduces a physical power penalty at peak utilization, it delivers profound,

system-level energy efficiency for modern duty-cycled applications that a fixed-depth pipeline cannot mathematically achieve.

4.6 Hardware Implementation and System-on-Chip (SoC) Integration

To transition the software-reconfigurable architecture from a simulated behavioral model into a physical silicon deployment, the verified adaptive CPU core was integrated into a complete System-on-Chip (SoC) ecosystem. This integration demonstrates the processor's capability to orchestrate external peripherals and execute bare-metal firmware natively on a physical FPGA fabric.

4.6.1 SoC Architecture and Peripheral Bus Routing

The adaptive CPU was wrapped within a centralized system bus architecture to facilitate communication with localized memory and peripheral IP blocks. To manage data routing effectively without bus contention, the SoC utilizes a hardcoded memory-mapped architecture. Figure 4.8 below shows the absolute physical address space allocated to the core memory structures and the memory-mapped I/O (MMIO) peripheral zones.

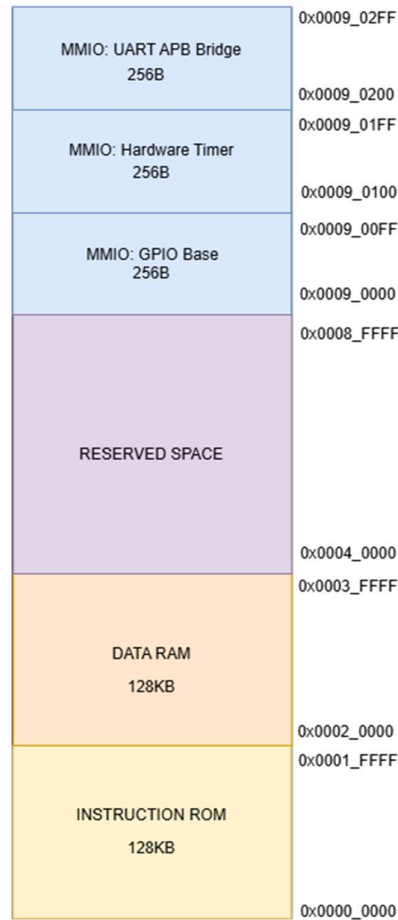


Figure 4.7: System-on-Chip (SoC) Physical Memory Map

The memory subsystem consists of the 128 KB Instruction ROM and 127 KB Data RAM. By allocating the peripherals to the 0x9000_0000 bounds, the centralized bus address decoder can efficiently route signals using minimal fabric logic, simply verifying the most significant nibble (MSB) of the 32-bit address request.

To enable physical board interactions, the SoC integrates three primary peripherals managed by the decoder:

1. **General Purpose Input/Output (GPIO):** Mapped to the MMIO base address (0x9000_0000), this module drives an 8-bit parallel output connected directly to the physical LEDs on the FPGA evaluation board, providing immediate visual verification of computational state.

2. **Hardware Timer:** Mapped to offset 0x0100, this module features programmable control and compares registers to generate precise, deterministic hardware interrupts, a critical feature for managing the duty-cycled IoT workloads.
3. **UART Bridge (APB Interface):** Mapped to offset 0x0200, a custom Advanced Peripheral Bus (APB) bridge was instantiated to translate the native CPU memory-mapped I/O requests into the standardized signaling required by the Lattice UART IP core, enabling full-duplex serial communication with a host PC.

4.6.2 Physical Board Demonstration

To prove the functional correctness of the physical silicon, three distinct bare-metal C applications were loaded into the FPGA Instruction ROM and executed natively. Each application was designed to stress-test specific microarchitectural components:

1. **Synchronous GPIO Blinker Application:** Validated basic instruction fetching, memory-mapped I/O routing, and the reliable decoding of custom Control and Status Register (CSR) instructions used to trigger hardware pipeline mode transitions.
2. **Pseudo-Random Number Generator (PRNG):** Validated the physical Arithmetic Logic Unit (ALU). By executing a 32-bit Xorshift algorithm bound strictly to hardware registers, the datapath was subjected to heavy logical shifts and XOR operations, confirming the structural correctness of the carry-chains under high toggle rates.
3. **Knight-Rider Sweep Application:** Validated control flow and conditional branching logic. The flawless physical execution of alternating bit-shifts,

threshold comparisons, and conditional jumps proved that the hardware hazard mitigation unit correctly resolves control dependencies in real-time execution.

The successful execution of three bare-metal applications below provides definitive, real-world proof of the processor's viability. Figure 4.9 captures the physical Lattice CertusPro-NX evaluation board executing the compiled firmware, with the 8-bit GPIO peripheral actively driving the onboard LEDs in real-time.

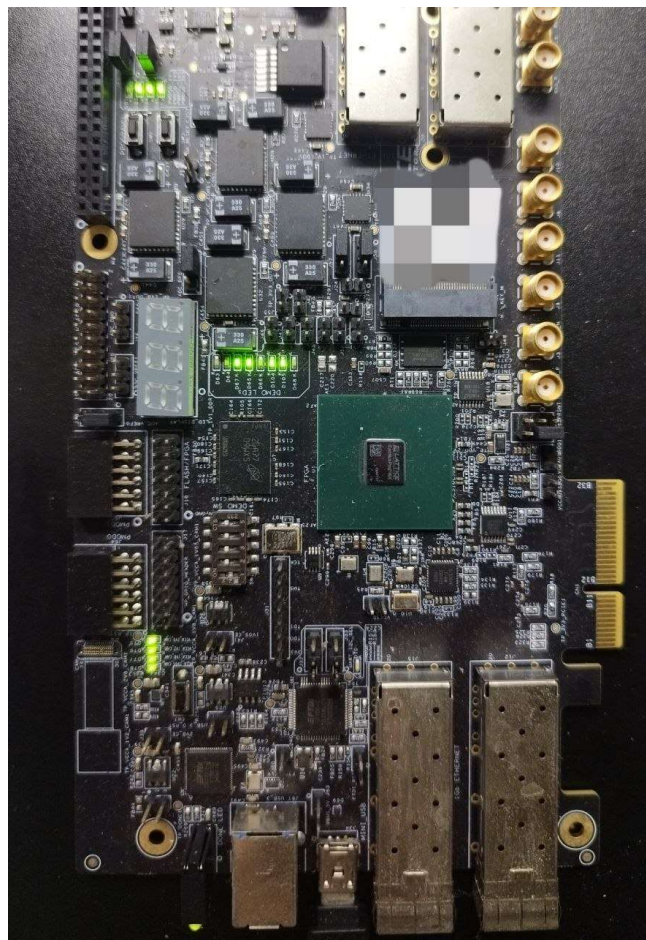


Figure 4.8: Physical Hardware Demonstration of the Lattice CertusPro-NX FPGA board

This physical demonstration satisfies the final hardware requirement of this research, proving that the software-reconfigurable RISC-V architecture functions

correctly not just in cycle-accurate simulation, but as a robust, fully deployed System-on-Chip (SoC).

4.7 Chapter Summary

This chapter traced the evolution of our RV32I RISC-V processor from a basic single-cycle design into a software-reconfigurable 3/5-stage architecture. To ensure a fair comparison, every version of the core was built and tested on the exact same Lattice CertusPro-NX FPGA. This allowed us to pinpoint the real-world trade-offs in hardware size, speed limits and power consumption.

Our physical implementation demonstrated that the adaptive core successfully resolves the traditional performance-power trade-off. When maximum throughput is required, the architecture operates seamlessly in its 5-stage mode at 50 MHz to deliver the highest instruction throughput for compute-intensive tasks, whereas a static 5-stage core remains trapped at this high-performance ceiling which wastes significant energy during the idle phases common in modern embedded systems.

By switching clock domains safely on the fly and power down unused pipeline registers, the adaptive core dynamically shifts into an efficient 3-stage mode to match immediate energy-efficient demands. This runtime elasticity slashes average dynamic power by up to 39.51% during typical IoT duty cycles. Ultimately, this chapter bridged the gap between theory and reality by successfully deploying the adaptive SoC onto CertusPro-NX Lattice FPGA board and validating its real-time reliability through the execution of bare-metal C firmware.

CHAPTER 5 CONCLUSION AND FUTURE WORK

5.1 Summary of Achieved Research Objectives

This research detailed the sequential development, verification, and physical hardware deployment of a RISC-V processor. The simulation and data fetched from FPGA results match our original goals, confirming that the project successfully met all its objectives.

The first primary objective was to design and implement an RV32I RISC-V processor core transitioning from a single-cycle baseline into a fixed 5-stage pipeline, and ultimately into an adaptive 3/5-stage runtime-configurable core. The goal was to achieve at least a 50% reduction in dynamic power consumption when operating in the 3-stage efficiency mode compared to a baseline fixed 5-stage pipeline. This objective was successfully achieved. As quantified in the power characterization phase, coarse-grained clock gating and operand isolation dropped the active dynamic power from 59.000 mW in the adaptive 5-stage mode down to 21.000 mW in the adaptive 3-stage mode. This yielded a massive 64.41% reduction in dynamic power consumption effectively surpassed the initial target.

The second objective is to verify the functional correctness and hazard mitigation mechanisms of both pipeline configuration via cycle-accurate simulation, targeting 100% ISA compliance against the Spike reference model across all architectural checkpoints within the official RISCOF framework for the RV32I and Zicsr specifications. This requirement was fully satisfied due to both the static 5-stage pipeline and adaptive 3/5-stage pipeline successfully passed all 39 architectural checkpoints within the RV32I RISCOF test suite. Furthermore, the memory signature dumps matched the Spike reference model byte-for-byte, confirming a 100% ISA

compliance rate and validating the structural integrity of data forwarding and hazard stall units.

Finally, the third objective aimed to implement and evaluate the complete System-on-Chip (SoC) on a Lattice CertusPro-NX FPGA, achieving timing closure at a target frequency of 50 MHz for the 5-stage mode and 20 MHz for the 3-stage mode. Post-route timing analysis confirmed the positive setup slack across both operational domains, achieving a maximum theoretical frequency of 57.531 MHz for the 5-stage configuration and 24.914 MHz for the 3-stage configuration, thereby supporting the intended dual-clock targets. By executing bare-metal micro-benchmarks and logging toggle rates via VCD trace annotation, the adaptive core demonstrated an average energy saving of 39.51% across typical workloads which validating the targeted efficiency range in a physical hardware environment.

5.2 Research Contributions and Key Findings

This research contributes to the field of low-power RISC-V design by providing a hardware-validated, adaptive microarchitecture that bridges the gap between high-throughput and energy-efficient operation. A primary contribution of this work is the development of a runtime-configurable datapath that dynamically adjusts pipeline depth. Unlike static cores that are permanently optimized for either performance or efficiency, this architecture enables the core to adapt its pipeline depth to real-time workload requirements without necessitating ISA modifications or compiler-specific optimizations.

Furthermore, this research establishes a rigorous methodology for physical power optimization and characterization. By implementing combinational operand isolation and coarse-grained clock gating, the design demonstrates that substantial

dynamic power savings can be realized in RTL-based FPGA implementations. These improvements were not merely simulated but were validated through a high-fidelity power profiling workflow that mapped RTL-level activity to physical FPGA routing using activity-annotated Value Change Dump (VCD) files. This workflow bridges the gap between theoretical power models and empirical hardware measurements.

Finally, this work contributes a standardized verification workflow for adaptive pipelines. By successfully applying the official RISCOF compliance framework against the Spike golden model, this project proves that dynamic reconfigurability does not compromise instruction-set architectural (ISA) correctness. This demonstrates that custom, adaptive microarchitectures can achieve industrial-standard reliability, providing a replicable model for future low-power RISC-V research.

5.3 Limitations and Constraints

To provide a transparent academic assessment, the microarchitectural and physical limitations of the current design methodology must be acknowledged. First, the dynamic power dissipation metrics presented in this research rely on theoretical models generated by the Lattice Power Calculator. While this is a highly accurate, industry-standard methodology for post-route power estimation, it does not represent direct physical measurement. Because the Lattice CertusPro-NX evaluation board has a highly integrated power delivery network, the core voltage rails could not be physically isolated. As a result, the exact power drawn from the processor cannot directly measure using shunt resistors and oscilloscopes.

Second, the processor architecture was strictly developed and verified against the RV32I Base Integer Instruction Set. Because the core lacks the standard RISC-V

‘M’ extension for dedicated hardware multiplication and division, it must rely entirely on software emulation libraries to resolve complex arithmetic. This microarchitectural constraint increases both the total instruction count and execution latency for computation-heavy workloads especially when related to multiplication and division workloads. While the adaptive pipeline handles these tasks reliably, adding dedicated hardware multipliers or dividers would improve the execution speed and reduce the energy-per-instruction for the heavy workloads.

Finally, it is important to note that switching pipeline modes introduces a slight latency penalty. Transitioning between the 3-stage and 5-stage depths requires a “pseudo-4th stage” to safely synchronize the clock domains and protect data integrity. As a result, the processor must briefly stall whenever it changes states. This overhead is negligible for long-running tasks, but it creates a strict throughput trade-off for any application that attempts to rapidly toggle the pipeline depth cycle-by-cycle.

5.4 Future Work

While the current architecture establishes a strong foundation for adaptive pipeline scaling, there are several clear pathways to expand its utility and performance in real-world applications. First, the instruction set architecture can be expanded beyond the RV32I base. By integrating the standard ‘M’ extension for hardware multiplication and division, it would eliminate software emulation overhead. These dedicated execution units would allow the 5-stage mode to handle multiplication-heavy and division-heavy workloads.

Besides that, the mode-switching mechanism can be fully automated. Currently, pipeline scaling relies on explicit software register writes, which requires the firmware programmer to manually define active and idle states. Future revisions

could integrate a hardware performance monitor to track metrics like pipeline stall conditions and instruction retirement rates. This monitor could drive automatic mode transitions, allowing the core to scale its own power consumption dynamically without needing compiler support or software intervention.

Finally, migrating the RTL design from an FPGA fabric to a standard-cell ASIC flow represents a critical next step for this architecture. While programmable logic inherently suffers from high static leakage, moving to a custom-routed silicon architecture effectively removes this overhead. Besides reducing baseline leakage, ASIC implementation enables true Dynamic Voltage and Frequency Scaling (DVFS). By actively scaling down the core voltage alongside the clock frequency during 3-stage operation, the processor can maximize its energy efficiency and comfortably meet the strict, ultra-low-power constraints of commercial IoT nodes.

REFERENCES

- [1] Ziyang He, “Design a Three-Stage Pipelined RISC-V Processor Using SystemVerilog,” 2022. Available: <https://kth.diva-portal.org/smash/get/diva2:1713647/FULLTEXT01.pdf>
- [2] A. Waterman, “Design of the RISC-V Instruction Set Architecture,” 2016. Available: <https://people.eecs.berkeley.edu/~krste/papers/EECS-2016-1.pdf>
- [3] K. Kırallı and C. B. Fidan, “Implementation of FPGA based 32-bit RISC-V processor,” *Engineering Science and Technology, an International Journal*, vol. 70, p. 102139, Aug. 2025, doi: <https://doi.org/10.1016/j.jestch.2025.102139>.
- [4] João Filipe Monteiro Rodrigues, “Configurable RISC-V softcore processor for FPGA,” 2019. Available: <https://docslib.org/download/76242/configurable-risc-v-softcore-processor-for-fpga-implementation>
- [5] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual Volume I: Unprivileged ISA Document Version 20190608-Base-Ratified,” 2019. Available: https://csg.csail.mit.edu/6.375/6_375_2019_www/resources/riscv-spec.pdf
- [6] Qin, Zhongyang. “Design and hazard solving of five-stage pipeline RISC-V processor structure,” 2024. Available: https://www.researchgate.net/publication/378435482_Design_and_hazard_solving_of_five-stage_pipeline_RISC-V_processor_structure
- [7] Z. Jin, H. Di, T. Hu, and P. Wang, “Real-Time Optimization of RISC-V Processors Based on Branch Prediction and Division Data Dependency,” *Applied Sciences*, vol. 15, no. 2, pp. 632–632, Jan. 2025, Available: <https://doi.org/10.3390/app15020632>.
- [8] D. Su, Y. Li, and B. Yuan, “SSR: A Stall Scheme Reducing Bubbles in Load-Use Hazard of RISC-V Pipeline,” *Advances in intelligent systems and computing*, pp. 559–565, Jan. 2020, Available: https://doi.org/10.1007/978-3-030-44041-1_50.
- [9] H. W. Kang and J. W. Choi, “basic_RV32s: An Open-Source Microarchitectural Roadmap for RISC-V RV32I,” *arXiv.org*, 2025. <https://arxiv.org/abs/2510.15887>

- [10] Jiongrui He, “Superscalar RISC-V Processor with Simd Vector Extension”, (2020). Available: <https://harvest.usask.ca/server/api/core/bitstreams/4f3f4b13-a26f-40ea-86f2-c8b607d6ea1f/content>
- [11] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [12] J. Yao, S. Miwa, H. Shimada, and S. Tomita, “Optimal pipeline depth with pipeline stage unification adoption,” *ACM SIGARCH Computer Architecture News*, vol. 35, no. 5, pp. 3–9, Dec. 2007, doi: <https://doi.org/10.1145/1360464.1360470>.
- [13] A. Djupdal, M. Sjölander, M. Jahre, S. Aunet, and T. Ytterdal, “Optimizing Energy Efficiency in Subthreshold RISC-V Cores,” *arXiv.org*, 2025. <https://arxiv.org/abs/2502.06588>
- [14] G. Qin, Y. Hu, L. Huang, and Y. Guo, “Design and Performance Analysis on Static and Dynamic Pipelined CPU in Course Experiment of Computer Architecture,” *2018 13th International Conference on Computer Science & Education (ICCSE)*, Aug. 2018, doi: <https://doi.org/10.1109/iccse.2018.8468729>.
- [15] B. V. V. Satyanarayana, N. Madhu and K. Srinivas, "Design and Implementation of High-Performance RISC-V Processor with Five Stage Pipeline on FPGA," *2025 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS), Erode, India, 2025*, pp. 68-75, doi: 10.1109/ICSSAS66150.2025.11081376.
- [16] L. Zhong and N. K. Jha, “Interconnect-aware high-level synthesis for low power,” *IEEE/ACM International Conf. on CAD*, Jan. 2002, doi: <https://doi.org/10.1145/774572.774588>.
- [17] O. Simola, “Physical implementation of a RISC-V processor,” *Aalto.fi*, Mar. 20, 2023. <https://aaltodoc.aalto.fi/items/efe24aaf-5415-4d45-b850-4c39510b9751/full>
- [18] Sallar Ahmadi-Pour, V. Herdt, and R. Drechsler, “The MicroRV32 framework: An accessible and configurable open source RISC-V cross-level platform for education and research,” *Journal of Systems Architecture*, vol. 133, pp. 102757–102757, Dec. 2022, doi: <https://doi.org/10.1016/j.sysarc.2022.102757>.

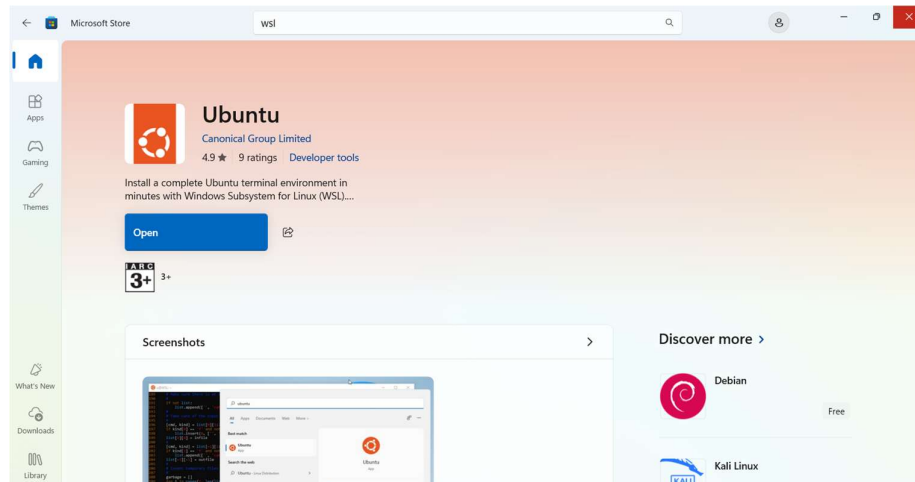
- [19] M. Jahnke, L. Bublitz, and Ulf Kulau, “Performance Evaluation of PicoRV32 RISC-V Softcore for Resource-Constrained Devices,” pp. 1–6, Oct. 2023, doi: <https://doi.org/10.1109/norcas58970.2023.10305479>.
- [20] D. Rossi et al., “A self-aware architecture for pvt compensation and power nap in near threshold processors,” *IEEE Design & Test*, vol. 34, no. 6, 2017.

APPENDICES

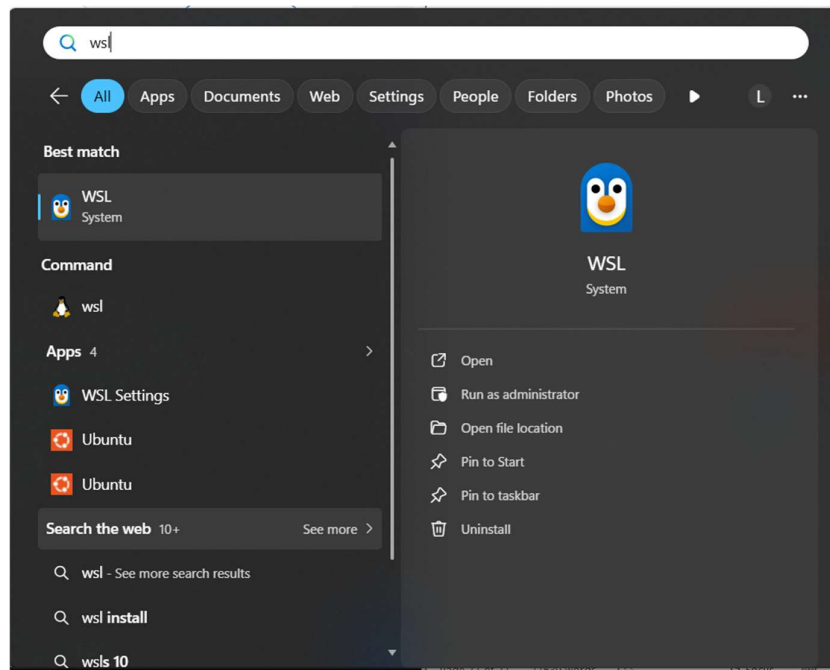
APPENDIX A WSL2 Environment Setup and RISCOF Toolchain Installation

WSL2 and Ubuntu Installation

1. Go to Microsoft Store and Install “Ubuntu”



2. Open WSL from the Windows searchbar



3. Once inside the Ubuntu terminal, update the package database with commands

```
sudo apt update && sudo apt upgrade -y
```

Toolchain and RISCOF Setup

1. Install the necessary tools.

```
sudo apt install autoconf automake autotools-dev curl python3 libmpc-dev  
libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf libtool  
patchutils bc zlib1g-dev libexpat-dev
```

2. Install RISCOF

```
pip3 install risc-of
```

3. Install the Spike ISA Simulator to act as the architectural reference

```
sudo apt install device-tree-compiler  
git clone https://github.com/riscv-software-src/riscv-isa-sim.git  
cd riscv-isa-sim && mkdir build && cd build  
../configure --prefix=/opt/riscv  
make && sudo make install
```

4. Initialize the DUT model folder and modify the `riscov\_my\_riscv\_soc.py` to point to the Verilator as executable engine and define the signature extraction logic.
5. Then, run the compliance tests suite based on the processor specification.

```
riscov run --config=config.ini --suite=riscv-arch-test/riscv32i --env=riscv-  
arch-test/libs
```

6. The python plugin is configured to monitor the `test\_finished` signal. Upon assertion, the framework executes a memory dump of the signature region and compares with the reference signature. Finally, comes out with a report with html website format to debug the failed cases.

APPENDIX B Open-Source Repository

The repository contains the complete evolutionary design of the adaptive RISC-V processor, organized into the following directories

1. ***Phase1_Single_Cycle***: Contains the baseline non-pipelined RV32I microarchitecture, including the fundamental Arithmetic Logic Unit (ALU), Instruction Decoder, and Register File.
2. ***Phase2_5_Stage***: Contains the classic 5-stage pipelined architecture, featuring complete data forwarding, hazard detection units, and branch prediction mechanisms.
3. ***Phase3_Adaptive***: Contains the flagship selectively adaptive pipeline, including the Pipeline Stage Unification (PSU) control logic and bypass multiplexers.



<https://github.com/MH-Electronic/RISC-V-Processor-Design>